Slovak University of Technology in Bratislava
Faculty of informatics and information technologies

Reg. No.: FIIT-182905-116160

Lukáš Častven

Designing Zero-Knowledge Proof Solutions in Ethereum ecosystem

Master's thesis

Thesis supervisor: Ing. Kristián Košťál PhD.

May 2026

Slovak University of Technology in Bratislava
Faculty of informatics and information technologies

Reg. No.: FIIT-182905-116160

Lukáš Častven

Designing Zero-Knowledge Proof Solutions in Ethereum ecosystem

Master's thesis

Study programme: Intelligent software systems

Study field: Computer Science

Training workplace: Institute of Computer Engineering and Applied Informatics

Thesis supervisor: Ing. Kristián Košťál PhD.

May 2026

Slovak University of Technology in Bratislava
Institute of Computer Engineering and Applied Informatics

Faculty of Informatics and Information Technologies
2025/2026

MASTER THESIS TOPIC

Author of thesis:	Bc. Lukáš Častven
Study programme:	Intelligent Software Systems
Study field:	Computer Science
Registration number:	FIIT-182905-116160
Student's ID:	116160
Thesis supervisor:	doc. Ing. Kristián Košťál, PhD.
Head of department:	Ing. Katarína Jelemenská, PhD.
Title of the thesis:	**Designing Zero-Knowledge Proof Solutions in Ethereum ecosystem**
Language of thesis:	English
Topic specifications:	Zero-knowledge proofs (ZKPs) are a new cryptographic primitive in applied cryptography with applications in multiple industries, including Web3, supply chains, and the Internet of Things. By verifying the authenticity of information without disclosing its content, ZKPs improve privacy, security, and efficiency in digital systems. Current use cases include decentralized identity (Worldcoin), private transactions (stealth address schemes or blockchains like Zcash and Monero), secure and scalable Layer-2s (ZkSync, Scroll) voting systems, IoT networks, and supply chain management. Examine existing solutions, proposals, and trends in this domain. Analyse a specific challenge discovered through the related work. Design a solution to address the challenge. Implement and test the solution on Ethereum (or a Layer-2) blockchain network. Evaluate and compare results with existing approaches. Discuss findings and contributions. Conclude with novelty, scientific findings, and future research directions.
Deadline for submission of thesis:	04. 05. 2026

Declaration

I hereby declare that I prepared this work independently, based on consultations and using the cited literature.

V Bratislave, 1.5.2026 .....

Lukáš Častven

Anotácia

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií
Študijný program: Inteligentné softvérové systémy

Autor: Lukáš Častven
Diplomová práca: Návrh riešení využívajúcich dôkazy s nulovou znalosťou v Ethereum ekosystéme
Vedúci diplomovej práce: Ing. Kristián Košťál PhD.
Máj 2026

Táto práca skúma návrh a implementáciu riešení využívajúcich dôkazy s nulovou znalosťou (ZKP) v rámci ekosystému Ethereum. Hlavným cieľom bolo za pomoci ZkVM odbremeniť nódy na exekučnej vrstve o potrebu znovavykonania nového bloku. Na docielenie bola použitiá lattice-based kryptografia, ktorá na rozdiel od eliptických kriviek, je post-kvantovo bezpečná a na rozdiel od haší, produkuje dôkazy s veľkosťou v desiatkach kilobajtoch a nie stovkach až tisícoch.

V práci sa podarilo navrhnúť architektúru a čiastočne implementovať prototyp lattice-based ZkVM. Implementovaný prototyp na porovnaniach výpočtu 100. Fibonacciho čísla vygeneroval dôkaz za 688,24 za využitia 12,34 GB RAM. V dlhšom EVM experimente systém bežal približne 9 hodín a využil 13,7 GB RAM. Výsledky ukazujú, že prístup je zatiaľ pomalší než etablované ZkVM riešenia, no súčasne naznačujú, že negatívny výsledok nie je spôsobený principiálym problémom, ale potrebuje optimalizacie. Čo sa týka veľkosti post-kvantovo bezpečných dôkazov, tak tie sú 4 až 5krát menšie, ako porovnané riešenia.

Annotation

Slovak University of Technology in Bratislava

Faculty of informatics and information technologies

Degree Course: Intelligent software systems

Author: Lukáš Častven

Master's thesis: Designing Zero-Knowledge Proof Solutions in Ethereum ecosystem

Supervisor: Ing. Kristián Košťál PhD.

May 2026

This thesis examines the design and implementation of solutions using Zero-Knowledge Proofs (ZKPs) in the Ethereum ecosystem. The main goal was to relieve execution-layer nodes from the need to re-execute every new block by means of a ZkVM. To achieve this, lattice-based cryptography was used, which, unlike elliptic-curve-based approaches, is post-quantum secure and, unlike hash-based approaches, can produce proofs in the order of tens of kilobytes rather than hundreds or thousands.

The thesis succeeded in designing the architecture and partially implementing a prototype lattice-based ZkVM. On the simple benchmark program, the prototype produced a proof in 688.24 seconds using 12.34 GB of RAM. In a longer EVM experiment, the system ran for approximately 9 hours and used 13.7 GB of RAM. The results show that the approach is currently slower than established ZkVM solutions, but they also suggest that this negative result is not caused by a fundamental limitation and that further optimization is needed. In terms of proof size, the post-quantum secure proofs are 4 to 5 times smaller than the compared solutions.

Table of contents

1 Introduction **1**

2 Analysis **5**

2.1 Ethereum 5

2.1.1 Consensus 6

2.1.2 Execution 6

2.2 Snarkification of Ethereum 7

2.2.1 Zero Knowledge Proofs 8

2.2.2 Snarkifying consensus 9

2.2.3 Snarkifying execution 9

2.3 ZkEVM 10

2.3.1 Challenges of EVM 10

2.3.2 Recursive proofs 11

2.3.3 IVC 12

2.3.4 First generation of ZkEVMs 13

2.3.5 Folding schemes 13

2.3.6 Instantiating IVC with folding scheme 15

2.3.7 New generation of ZkEVMs 16

2.3.8 Comparison 17

2.4 Lattice-based cryptography 19

2.4.1 Rings instead of matrices 20

2.4.2 Ajtai commitments 21
2.4.3 Lattices in ZK 22

3 Problem statement and research objectives 27

3.1 Problem statement 27
3.2 Research objectives 29
3.3 Scientific Questions 29

4 Solution Design 31

4.1 High level Architecture 32
4.1.1 The ZkVM IVC 32
4.1.2 Prover and Verifier Roles 36
4.2 Cryptographic Primitives 37
4.2.1 Incrementally Verifiable Computation (IVC) 37
4.2.2 Folding schemes and Nova 37
4.2.3 Customizable Constraint Systems (CCS) 38
4.2.4 Lattice-based hardness assumptions 39
4.2.5 LatticeFold 39
4.2.6 The wrapping SNARK SuperSpartan + Greyhound 39
4.2.7 Memory consistency check 40
4.2.8 LatticeFold+ 41
4.3 Designing the RISC-V circuit (F) 42
4.3.1 Field representations and arithmetization 42
4.3.2 State representation 42
4.3.3 Instruction decoding and execution 43
4.3.4 Memory consistency checks 45
4.4 IVC (F') 46
4.4.1 Enforcing IVC integrity 46

4.4.2 The on circuit NIFS verifier 48
4.4.3 Output generation 49
4.4.4 IVC verification 49
4.5 Final Wrapping SNARK 50
4.5.1 Wrapped statement 50
4.5.2 SuperSpartan layer 51
4.5.3 Greyhound layer 51
4.5.4 Soundness 52

5 Implementation **53**
5.1 Technology stack 54
5.2 Cryptography 55
5.2.1 Poseidon2 55
5.2.2 LatticeFold 56
5.3 RISC-V Emulator 56
5.4 Circuit synthesis 57
5.4.1 Poseidon2 constraint example 57
5.4.2 Self-aware NIFS constraints 60
5.5 Implementation optimizations 61
5.5.1 `arkworks-rs` upgrade 61
5.5.2 Reusing matrix structures 61
5.6 High level overview 62
5.6.1 Toolchain summary 64
5.6.2 Instruction coverage 64
5.6.3 Parameter set 65
5.6.4 CCS dimensions 66

6 Evaluation **69**

6.1 Fibonacci benchmark 70
6.2 Comparison with other ZkVMs 70
6.3 Why is the prototype slower? 73
6.4 Potential speed improvements 75
6.4.1 SIMD and GPU acceleration 75
6.4.2 More mathematical optimizations 76
6.5 Addressing the research questions 77

7 Conclusion **81**
7.1 Future Work 82

Resumé **85**
7.2 Motivácia práce 85
7.3 Vymedzenie problému 87
7.4 Návrh riešenia 89
7.5 Implementácia 91
7.6 Vyhodnotenie 93
7.7 Záver 95

Use of AI **97**
7.8 Document writing 97
7.9 Implementation 97
7.10 Evaluation 98

References **99**

List of Figures

2.1 Recursive proving 12
2.2 Visualization of the IVC chain [23] 13
2.3 IVC instantiation with folding scheme [29, 30] 16

4.1 Execution flow of the lattice-based RISC-V ZkVM IVC instantiation. 35

5.1 High-level overview of the implemented execution flow. 63

List of abbreviations used

CCS	Customizable constraint systems
EVM	Ethereum Virtual Machine
ISA	Instruction Set Architecture
NIFS	Non-Interactive Folding Scheme
NTT	Number Theoretic Transform
L1	Layer 1
L2	Layer 2
PCS	Polynomial Commitment Scheme
PoS	Proof of stake
p2p	peer-to-peer
SNARK	Succinct Non-interactive Argument of Knowledge
STARK	Scalable Transparent ARgument of Knowledge
R1CS	Rank-1 Constraint System
ZK	Zero Knowledge
ZkEVM	Zero Knowledge Ethereum Virtual Machine
ZkVM	Zero Knowledge Virtual Machine
ZKP	Zero Knowledge Proof

Chapter 1

Introduction

Ethereum, a decentralized blockchain platform launched in 2015, provides a Turing-complete execution environment for smart contracts and decentralized applications [1]. Its modular architecture comprises a proof-of-stake consensus layer (utilizing Gasper [2]) and an execution layer governed by the Ethereum Virtual Machine (EVM [3]). A primary challenge for Ethereum is scalability, as the need for all nodes to validate and re-execute every transaction limits throughput and can centralize the network by increasing resource demands.

"Snarkification", the integration of Zero-Knowledge Proofs (ZKPs) like SNARKs, offers a path to mitigate these scalability issues. Initiatives such as the Lean consensus [4] aim to snarkify consensus, allowing validators to verify compact proofs instead of entire state components. Similarly, Zero-Knowledge Ethereum Virtual Machines (ZkEVMs) seek to make EVM execution provable, enabling nodes to verify block execution via a single proof. Enshrining a ZkEVM into Ethereum's Layer 1 could further enhance scalability and simplify the creation of native rollups [5].

Zero-Knowledge proofs allow proving a statement's truth without revealing underlying information. These statements are about an element membership in NP language, thus an entire computations can be proven with this primitives, for example like computations done by the EVM, turning it into a ZkEVM. However, developing ZkEVMs is challenging due to the EVM's original design: complex opcodes, large word sizes requiring range proofs, its stack-based nature, and ZK-unfriendly storage mechanisms. Early solutions utilized recursive proofs to aggregate computation steps, leading to the first generation of Layer 2 ZkEVMs. Folding schemes further optimized this by compressing instances. The current trend involves Zero-Knowledge Virtual Machines (ZkVMs), especially those based on the simpler, register-based RISC-V ISA [6, 7], which is more amenable to ZK proof generation. Recent systematization work also shows that ZkVMs differ substantially in ISA choices, trace layouts, memory checking, and proving backends [8].

The majority of currently deployed ZkVMs and ZkEVMs employ hash-based STARKs [8, 9], which achieve post-quantum security through unstructured cryptographic hash functions but incur costs in proof size and verification complexity. A smaller portion of the ecosystem continues to rely on quantum-vulnerable elliptic curve assumptions, such as those underlying ECDSA or KZG commitments. Lattice-based cryptography presents a compelling alternative. Rather than relying on unstructured hashes, these schemes ground their post-quantum security in structured algebraic hard problems like SVP or SIS [10, 11, 12], potentially offering the efficiency and mathematical properties associated with algebraic constructions while maintaining resistance to quantum attacks.

This thesis investigates the design and implementation of a RISC-V ZkVM us-

ing lattice-based cryptography. A key application is proving Ethereum block execution, contributing to the network's scalability and alternative way of achieving post-quantum security. This work explores the practical construction of such a system and aims to assess whether lattice-based cryptography can offer performance comparable to classical ZKP methods while ensuring quantum resistance.

Document Structure

This work is structured as follows. Chapter 2 provides the necessary background and analysis of the current state of Ethereum, ZkVMs, ZkEVMs, and lattice-based cryptography. It begins with an overview of Ethereum's consensus and execution mechanisms 2.1, followed by an analysis of snarkification initiatives 2.2. The chapter then examines ZkEVMs 2.3, discussing the evolution from recursive proofs to folding-based systems and modern RISC-V ZkVMs. Finally, it introduces lattice-based cryptography 2.4, covering Ajtai commitments and recent developments in lattice-based zero-knowledge systems.

Chapter 3 defines the research gap regarding post-quantum non-hash-based ZkVMs and establishes the specific scientific questions and objectives that guide the solution design and implementation. It outlines the problem of the absence of a practical lattice-based RISC-V ZkVM and formulates the feasibility, performance, and long-running computation questions addressed by this thesis.

Chapter 4 details the architectural and cryptographic design of the proposed lattice-based RISC-V ZkVM. It presents the high-level architecture 4.1, specifies the cryptographic primitives 4.2, describes the construction of the RISC-

V step circuit 4.3, and then defines the IVC circuit 4.4 that connects execution proving with recursive folding.

Chapter 5 presents the prototype implementation of the lattice-based RISC-V ZkVM. It describes the technology stack 5.1 and the main implementation details, including the cryptographic integration, the RISC-V emulator, the CCS synthesis process, and selected implementation optimizations.

Chapter 6 provides an initial evaluation of the implemented components. It presents benchmark results, compares the prototype with selected existing ZkVMs, analyzes the main performance bottlenecks, and answers the research questions from both the design and implementation perspectives.

Chapter 7 concludes the work by summarizing the design and implementation achievements and outlining directions for future work, including alternative computation models, hardware acceleration, improved lattice-based folding backends, and architectural optimization.

Chapter 2

Analysis

This chapter reviews the background needed for the proposed system. It first summarizes Ethereum's consensus and execution model, then explains how snarkification and ZkEVMs change block verification, and finally introduces the lattice-based cryptographic primitives relevant to the design.

2.1 Ethereum

Ethereum is a decentralized, open-source blockchain with smart contract functionality [1]. Conceptualized by Vitalik Buterin in 2013/2014, it went live in 2015, aiming to build a platform for decentralized applications that adds Turing complete execution environment to blockchain [13]. Ethereum enables developers to create and deploy decentralized applications and crypto assets. The Ethereum network is comprised of two layers, two peer-to-peer networks, consensus layer and execution layer.

2.1.1 Consensus

Consensus on Ethereum is achieved with proof-of-stake (PoS) consensus mechanism called Gasper [2]. PoS system relies on crypto-economic incentives, rewarding honest stakers (people who put economic capital in the network) and penalizing malicious ones.

Stakers, or also called validators, propose new blocks. Validator is selected for a block proposal pseudo-randomly from the pool in each slot. A slot is some time amount (as of writing of this work it is 12 seconds on Ethereum mainnet), in which the pseudo-randomly chosen validator can propose new block. The software creating the new block is called consensus client. Proposer's consensus client requests a bundle of transactions from the execution layer 2.1.2, wraps them into a block and gossips (sends) the new block to other participants over the consensus p2p network. The rest of the validator pool can in 32 slot (one epoch) attest to that new block's validity. In order for a block to be finalized, it must be attested by a super-majority, which is 66% of the total balance of all validators [3].

2.1.2 Execution

Software operating on the execution layer is called execution client. Nodes on execution layer hold the latest state and database of all Ethereum data. These clients gossip incoming transactions over the execution layer p2p network, and each stores them in a local mempool. Once they are put inside a block, each transaction is executed in Ethereum-Virtual-Machine (EVM). EVM is a stack based virtual machine operating 256 bit words which executes smart contract code. Ethereum is a decentralized state machine, with rules defined by the EVM. EVM can be thought of as a function. Given a valid

state and valid set of transactions[1], outputs next valid state:

$$F(state_n, T) = state_{n+1}$$

Thus, Ethereum's state transition function is described by the EVM. This function must be executed by each execution node for each new block in order to keep up with the current chain state [14].

2.2 Snarkification of Ethereum

As discussed in sections on consensus (2.1.1) and execution layer (2.1.2), nodes on each layer must expend compute resources to validate the consensus data of new blocks or re-execute all transactions within them to verify and maintain the blockchain's state. These computational demands limit the scalability of the entire network.

Ethereum's decentralized node network includes not only participants with capable servers, but also hobbyists, who use home internet connections and less powerful machines. Naively attempting to scale the network, for instance, by decreasing slot time, or increasing block size may raise bandwidth and node requirements, thereby ousting less capable nodes and thus hindering Ethereum's decentralization.

Snarkification refers to the process of integrating a specific type of Zero Knowledge Proofs (ZKPs) called SNARK (Succinct Non-interactive Argument of Knowledge) in order to offload a computation to one entity (one node, or a cluster that is a small subset of the whole network), which performs the

[1] Transaction and state-transition validity are verified by execution clients; consensus clients coordinate agreement on blocks containing execution payloads 2.1.1.

computation and generates a proof of its validity. This proof can then be verified by the rest of the network at fractional compute cost [5].

2.2.1 Zero Knowledge Proofs

Zero-Knowledge Proofs are cryptographic primitives that allow a prover to convince a verifier of a statement's truth without revealing any information beyond the statement's validity itself. The statement typically concerns membership in an NP language/relation [15]. The membership statements are usually named ZK circuits. Traditionally, ZKPs are interactive protocols where a prover, who knows a private witness for a statement (e.g., a solution to a problem), aims to convince a verifier of this knowledge. Through a series of back-and-forth interactions, the verifier becomes convinced of the prover's claim (if true) without learning the witness itself.

Interactive proofs where the verifier's messages consist only of random challenges are known as public-coin protocols. The Fiat-Shamir heuristic [16] can transform such public-coin interactive proofs into non-interactive proofs by replacing the verifier's random challenges with challenges derived from a cryptographic hash function applied to the protocol's transcript. This non-interactivity allows a single generated proof to be validated by any number of verifiers.

A SNARK is a specific type of ZKP. The 'Succinct' aspect implies that proof sizes are very small (e.g., polylogarithmic in the size of the witness or statement) and verification is faster than re-executing the original computation, often polylogarithmic in the computation's complexity [17].

2.2.2 Snarkifying consensus

The Lean consensus [4] is a research initiative, introduced by Justin Drake at Devcon 2024 [18], aimed at redesigning and improving Ethereum's consensus mechanism. A key aspect of this proposal involves leveraging SNARKs to make consensus layer state transitions provable, in near real-time. Instead of each validator independently re-validating all components of a new consensus state, they would verify a single, compact SNARK. Based on this proof's validity, they can accept or reject the proposed state update.[2]

2.2.3 Snarkifying execution

Ethereum's execution state transitions are dictated by the rules of the EVM. To reduce the computational burden of verifying execution using ZKPs, the EVM's execution logic needs to be provable with a ZKPs, creating a ZkEVM. This would change the state transition function (2.1.2) to:

$$F(state_n, T) = (state_{n+1}, \pi) \tag{2.1}$$

Where π is the proof of the state transition's validity. Therefore, verifying a block's execution state transition would involve verifying this single proof, rather than re-executing all transactions within the block. Furthermore, an increase in block size would have a little to no impact on verification times for nodes, as proof verification complexity grows much slower than the computation being proven. This could allow for larger block sizes without risking the exclusion of nodes with modest computational capabilities due to their weaker computational capabilities, thereby supporting scalability.

[2]The entity proposing the state update, such as a block proposer, would be responsible for generating and submitting this proof.

Another approach is to "enshrine" a ZkEVM directly into the Ethereum protocol (L1). This offers similar benefits in terms of L1 state transition verification, as the native EVM could delegate execution proving to this enshrined ZkEVM. In addition, an enshrined ZkEVM could support and simplify the creation of native rollups [5]. These would be Layer 2 solutions that could leverage the L1's enshrined ZkEVM for verifying their own state transitions, which are themselves batches of EVM-compatible transactions. The correctness of these L2 transaction batches would thus be directly enforced by the L1 execution layer through ZK proof verification. This could lead to faster, settlement for these L2s on L1, which in turn could simplify composability between L1 and these native L2s, as well as among different native L2s.

2.3 ZkEVM

ZkEVM, as mentioned in 2.2.3, is a modified EVM, which given state and list of transactions produces next state and a proof π attesting to correctness of that execution. The primary use case of ZkEVMs today is enabling "ZK rollups", L2s which execute a bundle of transactions offchain, generate a proof and submit the bundle with the proof on the L1, where only the proof is validated. A second use case, though not used today, is to enshrine ZkEVM into EVM as described in 2.2.3.

2.3.1 Challenges of EVM

The EVM was not designed with ZK taken into consideration:

1. Many EVM opcodes are complex and expensive to prove with ZK. This has led to different types of EVM compatibility among ZkEVMs [19].

2. 256 bit word size means that ZK systems working over prime fields must include range proofs, which increases the ZkEVM complexity.
3. It is harder to prove a stack based VM. For example, Starkware developed Cairo [20], a register based model in order to implement its ZkEVM, which requires a custom smart contract compiler.
4. EVM storage uses Merkle Patricia Tries with Keccak, which are not ZK friendly and have huge proving overhead.

Proving the entire EVM within a single circuit (one proof) is computationally and economically unfeasible. Such a proof would be several megabytes in size, making it too expensive to store on L1, let alone prove.

2.3.2 Recursive proofs

Recursive proofs combine benefits of ZKP systems with fast prover times (e.g., those based FRI [21]) and systems with short proofs (like Groth16 [22]). The idea is to produce a proof of a knowledge of a proof.

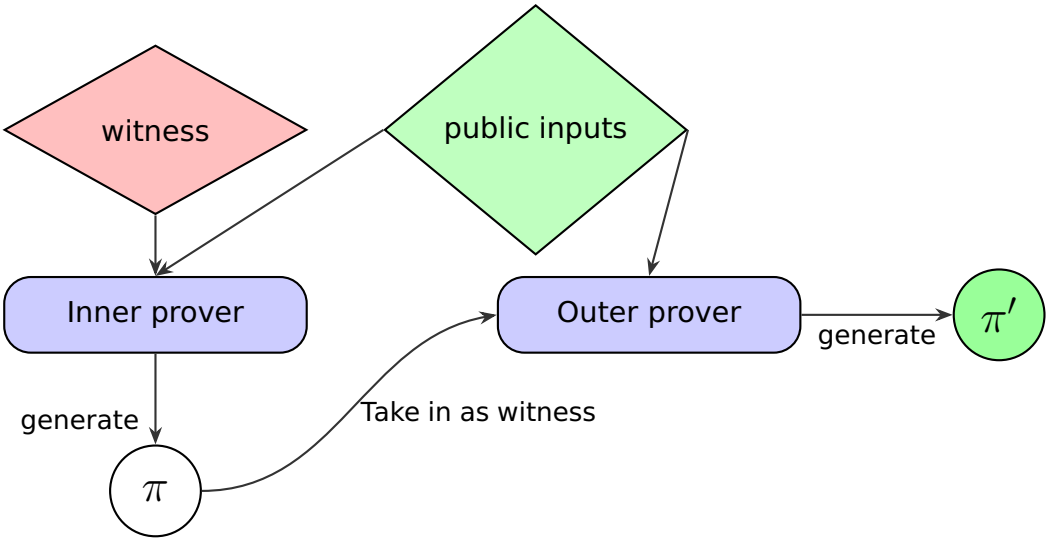

Figure 2.1: Recursive proving

The recursive ZKP is an instantiation of 2 step Incrementally Verifiable Computation (IVC) [23].

2.3.3 IVC

IVC is a cryptographic primitive for proving the correctness of an iterated and incremental computation (such as an EVM or RISC-V microprocessor execution) [23, 24].

Let F be a step function (e.g., representing the execution of a single instruction). The output state of step i is fed as input into step $i + 1$, along with a proof π_i attesting to the validity of the history up to that point.

The final proof π_n serves as a cryptographic certificate. It attests that the prover possesses a valid sequence of private witnesses (non-deterministic advice) $u_1, \ldots, u_n$, such that applying the transition function F sequentially

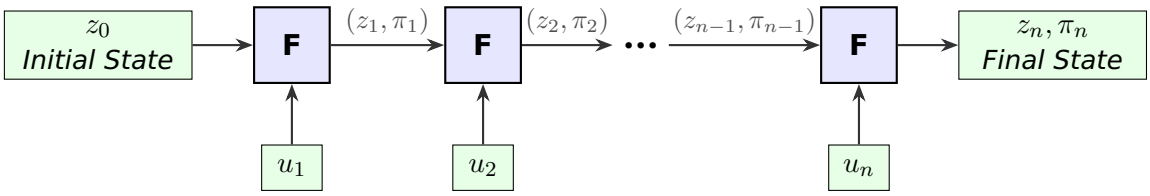

Figure 2.2: Visualization of the IVC chain [23]

starting from the initial state z_0 results in the final public state z_n. The first practical implementation of IVC was achieved using recursive SNARKs [25].

2.3.4 First generation of ZkEVMs

In the 2022 to early 2024 first generation of ZkEVMs was successfully deployed. Thanks to recursive proving, it was feasible for these ZkEVMs to be created. They first prove the EVM execution, and then prove knowledge of an EVM proof inside another circuit. This enabled parallelization of proving and reduced the size and complexity of the final proof submitted to L1. The proving system landscape of this period was strongly influenced by PLONK [26], while recursive systems such as Plonky2 [27] and Halo2 [28] became practical examples of this direction. In a broader system view, this period also laid the foundations for the more modular zkVM architectures surveyed in [8]. Notable references include Scroll, zkSync Era, Polygon ZkEVM.

2.3.5 Folding schemes

Even though recursion proofs enabled teams to build ZkEVMs, they are not without their flaws. The prover needs to have a circuit which contains the whole verification algorithm of another proof system. It must verify expen-

sive evaluation proofs for polynomial commitments.

Folding removes the need to verify a full SNARK inside the circuit, replacing it with a cheap evaluation of a random linear combination. It still requires a verifier circuit, but a much cheaper one. There are no FFTs, only multi-exponentiations, which do not require large memory overhead [29].

It compresses two instances into one. Folding prover will fold two instances, with corresponding witnesses, and produce single instance, s.t., if it is correct, it is implied that also the two original instances were correct [29].

Given a relation R, which defines valid instance-witness pairs (u, w) such that $R(pp, u, w) = 1$:

- pp = public parameters,
- u = instance (public inputs, commitments),
- w = witness (private data).

For example, R is an equation, u are coefficients and w are concrete values that satisfy the equation.

A folding scheme for a relation R is an interactive protocol where the prover P and verifier V reduce two instances u and U into a single instance U'.

Let the incoming step be denoted by the pair (u_i, w_i) and the running accumulator be denoted by (U_i, W_i). The folding protocol proceeds as follows:

1. P has $(u_i, w_i) \in R$ and $(U_i, W_i) \in R$.
2. V has u_i and U_i (but does not know the witnesses).

The result of their interaction is a new folded instance U_{i+1} (known to both) and a new folded witness W_{i+1} (known only to P), such that $(U_{i+1}, W_{i+1}) \in$

R.

Thus, instead of verifying two distinct instances (u_i and U_i), the verifier only needs to check the single folded instance U_{i+1} at the very end of the computation.

Folding scheme can be made non-interactive by using Fiat-Shamir heuristic [16].

2.3.6 Instantiating IVC with folding scheme

IVC instantiation via folding schemes requires wrapping the base computation F within an augmented function F', where F is the "application" circuit [29, 24, 30]. The function F' is structured to perform two primary tasks:

1. **Base computation:** F' executes the original state transition $F(z_{i-1}, u_i) \rightarrow z_i$, where z_{i-1} is the state from the previous step and u_i is non-deterministic advice for step i. This corresponds to verifying the correct execution of one step of the underlying computation (e.g., a single RISC-V instruction cycle).

2. **IVC maintenance:** This task ensures the integrity of the recursive proof chain. F' takes as non-deterministic advice the full state from the previous step: $(i, z_0, z_{i-1}, U_{i-1})$. It then performs two critical checks:

   - **Consistency check:** F' re-computes the hash of the input state and asserts that it matches the public input h_{i-1}. This binds the current step's execution to the correct history.

   - **Recursive verification:** F' executes the verifier logic of a Non-Interactive Folding Scheme (NIFS). It takes the running accumu-

lator U_{i-1}, the instance u_{i-1} (an instance from the step $i-1$) and claimed new folded accumulator U'_i. After recomputing U_i, the check passes if $U_i = U'_i$.

Finally, F' outputs a new hash h_i, committing to the updated state $(i+1, z_0, z_i, U_i)$, which serves as the public input for the next step [29, 24, 30]. Figure 2.3 illustrates this data flow.

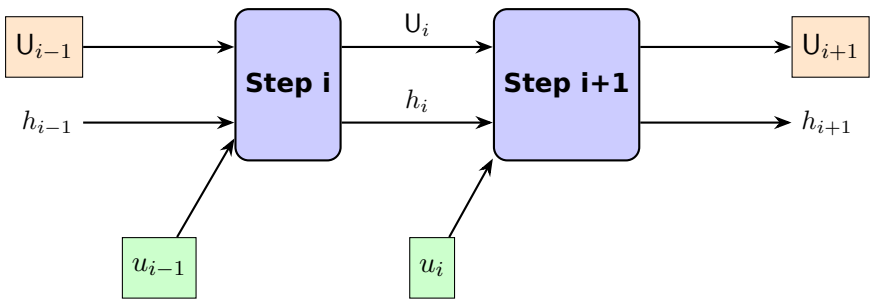

Figure 2.3: IVC instantiation with folding scheme [29, 30]

2.3.7 New generation of ZkEVMs

Currently, the industry is moving towards ZkVMs, dominantly RISC-V ZkVMs. RISC-V is an open-source Instruction Set Architecture (ISA) [8, 6, 7]. RISC-V's simpler, reduced instruction set is better suited for ZK proof computations. Its register-based architecture is generally more efficient for proving than the EVM's stack-based architecture. Another benefit is the mature tooling ecosystem, debuggers, and libraries. This allows developers to write provable programs in familiar languages like Rust, C, and C++. RISC-V is a general purpose ISA, which opens possibilities for proving not just blockchain applications. Due to this, an EVM written in a language that can be compiled into RISC-V, can be executed inside a ZkVM and together this

combination creates a ZkEVM.

2.3.8 Comparison

The only sensible comparison method for ZkEVMs is to benchmark them on proving Ethereum blocks. EthProofs [31] is a portal where ZkEVM teams can join and be measured on the most important thing, the proving times of Ethereum blocks. As of writing this work, around 75% of blocks are proven under 10 seconds.

Table 2.1 compares selected zkVMs relevant to this thesis. Airbender, ZisK, Pico, and OpenVM 2.0 were active on EthProofs at the time of writing, while SP1 was listed there as inactive and Nexus zkVM was not listed. Because these systems expose different final proof artifacts, compressed recursive proofs in some cases and wrapped SNARKs in others, proof size figures should be interpreted only as approximate.

The leading active systems in this comparison are STARK-based [9], but their internal architecture is not uniform. Airbender uses AIR constraint system with a DEEP-STARK/FRI pipeline. ZisK uses PIL2 as a state machine arithmetization with a STARK prover. Pico uses AIR with modular STARK and CircleSTARK backends. OpenVM is also AIR-based [32], but organizes much of the machine through modular chips, buses, interactions, and lookup/range check relations, and then wraps the result for Ethereum with Halo2 on BN254. Nexus zkVM 3.0 is AIR-based and uses lookup/logup relations over the Stwo Circle-STARK prover. SP1 is again somewhat different. Its Hypercube stack is AIR-based, but the base proof system is better described as a polynomial-IOP/STARK construction built from subprotocols such as Jagged PCS and Basefold rather than as a conventional univariate STARK pipeline

[33]. At the same time, these systems also differ in ISA design, trace structure, memory organization, and recursion strategy, which complicates direct performance comparison [8].

	Airbender	**ZisK**	**Pico**	**SP1**	**Nexus**	**OpenVM**
Organization	Matter Labs	SilentSig	Brevis	Succinct	Nexus	Axiom
EthProofs hardware	8×RTX 5090	24 GPU	16×RTX 5090	n/a	n/a	16× RTX 5090
EthProofs time	5.9s	8.3s	9.8s	n/a	n/a	4.3s
Proof size	∼1MB	∼260KB	∼1MB	260B / 868B	n/a	277 KB
ISA	RV32IM	RV64IMA	RV32IM	RV64IM	RV32IM	RV32IM
Constraint system	AIR	PIL2	AIR	AIR	AIR + logup	AIR
Base zk system	DEEP-STARK/FRI	STARK	STARK	Hypercube	Circle STARK	STARK + FRI
Wrapper	SNARK	PLONK/FFLONK	Groth16	Groth16 PLONK	none documented	Halo2/BN254

Table 2.1: Comparison of selected zkVMs relevant to this thesis. EthProofs hardware and proving-time entries are shown where publicly available.

2.4 Lattice-based cryptography

Lattice-based cryptography has emerged as a significant area of research, particularly due to its potential for constructing cryptographic primitives resistant to attacks by quantum computers. Unlike traditional public-key cryptosystems such as those based on elliptic curves, which are vulnerable to Shor's algorithm on a quantum computer, many lattice-based constructions are believed to maintain their security in a post-quantum world. The security of these schemes often relies on the presumed hardness of certain computational problems on lattices, such as the Shortest Vector Problem (SVP) [34, 11, 10].

A lattice L is a discrete subgroup of $\mathbb{R}^n$, typically represented as the set of all integer linear combinations of a set of linearly independent basis vectors $B = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n\}$ where $\mathbf{b}_i \in \mathbb{R}^n$. That is:

$$L(B) = \left\{ \sum_{i=1}^{n} x_i \mathbf{b}_i \mid x_i \in \mathbb{Z} \right\} \tag{2.2}$$

The dimension of the lattice is n. Many cryptographic constructions are built upon the difficulty of solving problems like finding the shortest non-zero vector in a lattice (SVP) or finding the lattice point closest to a given target vector (CVP), especially in high dimensions.

One of the pioneering works in this field was by Miklós Ajtai in 1996, who introduced a cryptographic construction whose security could be based on the worst-case hardness of lattice problems [34]. This provided a stronger theoretical foundation for cryptosystems using lattices.

Lattice-based cryptography offers several advantages:

- **Post-quantum security:** As mentioned, this is a primary driver for its adoption.
- **Efficiency:** Some lattice-based operations can be computationally efficient, particularly those involving matrix-vector multiplications over small integers [11].
- **Versatility:** Lattices have been used to construct a wide array of cryptographic primitives, including public-key encryption, digital signatures, and importantly for this thesis, vector commitment schemes [11].

2.4.1 Rings instead of matrices

In lattice-based cryptography, it is more efficient to use polynomial ring elements, instead of the traditional matrix representations. In these constructions, operations are performed in the quotient ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, where polynomials are considered modulo $x^n + 1$. This polynomial modulus induces a negacyclic structure. Multiplying by x corresponds to a right rotation of coefficient vectors, with a sign inversion when elements wrap around [35]. The ring-based approach provides significant advantages over unstructured lattices, storing a ring element requires only n coefficients rather than n^2 matrix entries, yielding key sizes that are an order of magnitude smaller [36]. Furthermore, polynomial multiplication can be computed efficiently in $O(n \log n)$ time using the Number Theoretic Transform (NTT), compared to the $O(n^2)$ complexity of matrix vector multiplication [37]. These efficiency gains, combined with the worst-case hardness guarantees inherited from Ring-LWE [37], have made ring-based schemes the preferred approach for practical lattice cryptography.

2.4.2 Ajtai commitments

A commitment scheme is a cryptographic primitive that allows a party (the prover) to commit to a value while keeping it hidden from another party (the verifier), with the ability to reveal the committed value later. The scheme must satisfy two main properties:

- **Hiding:** The verifier cannot learn the committed value from the commitment itself before the reveal phase.
- **Binding:** The prover cannot change the committed value after the commitment phase.

The Ajtai commitment scheme from Ajtai's 1996 work [34], is a lattice-based commitment scheme. Its security is based on the hardness of the Shortest Integer Solution (SIS) problem [34, 11, 38].

The SIS problem is defined as follows: Given a random matrix $A \in \mathbb{Z}_q^{n \times m}$ (where q is a modulus, n is the row dimension, and m is the column dimension, with $m > n \log q$), find a non-zero integer vector $\mathrm{s} \in \mathbb{Z}^m$ with small norm (i.e., $s_i \in s; ||s_i|| \leq \beta$, for some bound β) such that:

$$A\mathrm{s} = \mathbf{0} \pmod{q} \tag{2.3}$$

Finding such a short vector s is believed to be computationally hard for appropriate parameters.

- **Binding:** The binding property relies on the SIS assumption. If a prover could find two different openings (e.g., s and s′ where $\mathrm{s} \neq \mathrm{s}'$) for the same commitment $\mathbf{t}$, such that $A\mathrm{s} = \mathbf{t} \pmod{q}$ and $A\mathrm{s}' = \mathbf{t} \pmod{q}$, then $A(\mathrm{s} - \mathrm{s}') = \mathbf{0} \pmod{q}$. If $\mathrm{s} - \mathrm{s}'$ is a short non-zero vector,

this would imply a solution to the SIS problem for matrix A.

- **Hiding:** The hiding property is based on the Module Learning With Errors (MLWE) assumption. Informally, the commitment uses randomness, so the published value does not reveal the committed vector. Under MLWE assumptions, the resulting distribution is computationally indistinguishable from uniform to any efficient adversary [12].

A useful feature of some Ajtai commitments is that the size of the commitment $\mathbf{t}$ does not grow with the dimension of the message $\mathbf{s}$, though the message components themselves must be small [12].

2.4.3 Lattices in ZK

Lattice-based cryptography presents a compelling avenue for advancing zero-knowledge proof systems, due to its strong security foundations against quantum computers. This makes them a promising candidate for future-proofing ZK solutions.

LaBRADOR: Compact proofs for R1CS from Module-SIS

LaBRADOR, introduced by Beullens and Seiler [39], is a lattice-based proof system designed for Rank-1 Constraint Systems (R1CS). An advantage of LaBRADOR is its ability to generate very compact proof sizes, particularly for large circuits, making it more efficient in terms of proof transmission and storage compared to many other post-quantum approaches. For instance, LaBRADOR can prove knowledge of a solution for an R1CS modulo $2^{64} + 1$ with 2^{20} constraints with a proof size of only 58 KB.

However, a notable drawback of LaBRADOR is that its verifier runtime is linear in the size of the R1CS instance. This means that while the proofs

are small, the verification process is not succinct, which can limit its applicability in scenarios requiring fast, sublinear verification. Despite this, LaBRADOR's compact proof generation makes it a candidate as an "outer layer" or wrapper. It can be used to prove the correctness of a (potentially much larger) proof generated by another succinct, but perhaps less compact, proof system, thereby achieving overall succinctness with post-quantum security.

Greyhound: Fast Polynomial Commitments from Lattices

Building upon the strengths of LaBRADOR, Nguyen and Seiler proposed Greyhound, an efficient polynomial commitment scheme (PCS) based on standard lattice assumptions [40]. Greyhound leverages LaBRADOR as a core component to achieve succinct proofs of polynomial evaluation.

The construction involves a three-round protocol for proving evaluations for polynomials of bounded degree N with a verifier time complexity of $O(\sqrt{N})$. By composing this with the LaBRADOR proof system, Greyhound obtains a succinct proof of polynomial evaluation that also has a sublinear verifier runtime. For large polynomials (e.g., degree up to $N \approx 2^{30}$), Greyhound produces evaluation proofs of approximately 53KB. This demonstrates the practical utility of using LaBRADOR to compress proofs for more complex cryptographic primitives like polynomial commitments.

LatticeFold and LatticeFold+

In the domain of folding schemes, which are techniques for building efficient recursive SNARKs, Boneh and Chen introduced LatticeFold. LatticeFold addressed the challenge of maintaining low-norm witnesses through multiple

rounds of folding by employing a sumcheck-based range proof to ensure extracted witnesses remained valid [41].

More recently, Boneh and Chen proposed LatticeFold+ [42], an improved lattice-based folding protocol. LatticeFold+ enhances its predecessor in several key aspects: the prover is substantially faster (five to ten times), the verification circuit is simpler, and the folding proofs are shorter. These improvements are achieved through two main lattice techniques:

1. A new, purely algebraic range proof that is more efficient than the bit-decomposition approach used in LatticeFold.
2. The use of double commitments (commitments of commitments) to further shrink proof sizes, along with a new sumcheck-based transformation for folding statements about these double commitments.

LatticeFold+ aims to be competitive with, or even outperform, pre-quantum folding schemes like HyperNova [30] while offering plausible post-quantum security.

Neo

More recently, Nguyen and Setty proposed Neo [43], a lattice-based folding scheme for CCS over small prime fields. This is particularly relevant to the design space of this thesis, because CCS is also the chosen arithmetization here and the implementation works over a small field.

Neo can be viewed as bringing the HyperNova style of folding into the lattice-based setting while avoiding some of the overheads of LatticeFold's cyclotomic-ring formulation. In particular, the paper emphasizes native support for small fields such as Goldilocks, a single sum-check invocation over

an extension of a small prime field, and a new folding friendly Ajtai commitment scheme with pay-per-bit commitment costs. These properties make Neo one of the closest known lattice-based systems to the architecture targeted in this work.

Chapter 3

Problem statement and research objectives

Based on the analysis of the current state of ZkVMs and the theoretical properties of lattice-based cryptography, this chapter defines the specific problems addressed by this thesis. It outlines the research gap regarding post-quantum non hash-based ZkVMs and establishes the specific scientific questions and objectives that guide the subsequent solution design, implementation and evaluation.

3.1 Problem statement

The integration of ZKPs into blockchain architectures, particularly Ethereum, offers a path toward massive scalability. As detailed in the previous chapter, the industry standard has converged on ZkVMs that execute instruction sets like RISC-V. These systems allow developers to write programs in more familiar languages (such as Rust or C++) and prove their correct execution.

Some currently deployed ZkEVMs and ZkVMs rely on cryptographic hardness assumptions, specifically the Discrete Logarithm Problem in elliptic curve groups, that are vulnerable to attacks by sufficiently powerful quantum computers. Majority of ZkVMs use hash-based STARKs [9], which offer a post-quantum alternative, but they incur significant costs in terms of proof size or verification complexity compared to SNARKs based on specific algebraic structures.

Lattice-based cryptography presents a promising third path. It offers post-quantum security based on the hardness of the Shortest Vector Problem and the Module Short Integer Solution [34, 11]. Furthermore, lattice-based commitment schemes possess additive homomorphic properties that make them theoretically suitable for highly efficient folding schemes, such as LatticeFold [41].

Despite these theoretical advantages, a significant gap exists in the literature and open-source ecosystem, **there is currently no practical solution of a general-purpose RISC-V ZkVM built entirely upon lattice-based primitives.**

The problem addressed by this thesis is the lack of a concrete instantiation of a RISC-V ZkVM that leverages lattice-based folding schemes. It remains unproven whether the theoretical efficiency of lattice folding translates to practical performance for complex computations like a VM. Without such an implementation, it is impossible to accurately compare the performance trade-offs between lattice-based approaches and the elliptic curve architectures. This difficulty is reinforced by the fact that existing ZkVMs already span a highly heterogeneous design space across the ISA, VM, and proving layers [8].

3.2 Research objectives

The primary objective of this work is to design and implement a functional prototype of a RISC-V ZkVM using lattice-based cryptography, specifically leveraging the LatticeFold protocol for instantiating Incrementally Verifiable Computation of a RISC-V VM. This implementation aims to serve as a proof-of-concept for post-quantum execution environment for proving Ethereum blocks. The specific objectives are defined as follows:

- **Architectural Design:** Design a system architecture for a ZkVM that maps the RISC-V instruction set architecture to the constraints of the LatticeFold folding scheme. This involves defining the arithmetization of 32 bit CPU logic into a Customizable Constraint Systems (CCS) [44] structure, suitable to be used by LatticeFold.
- **Implementation:** Implement the proposed design using the Rust programming language, integrating with the available LatticeFold libraries.
- **Comparative evaluation:** Evaluate the performance of the implemented lattice-based ZkVM. The goal is to benchmark the system against establishing metrics for proving time and memory usage, comparing these results with theoretical expectations and, where applicable, existing non-post-quantum baselines.

3.3 Scientific Questions

To achieve the stated objectives, this thesis seeks to answer the following scientific questions:

1. **Feasibility of lattice-based instantiation:** *Is it feasible to instantiate*

a full RISC-V execution environment using CCS compatible with the LatticeFold protocol?

2. **Performance trade-offs:** *How does the performance of a lattice-based folding scheme compare to established approaches of proving a general-purpose VM execution?*

3. **Suitability for long-running computations:** *Does the noise growth inherent in lattice-based cryptography hinder the ability to prove long execution traces, such as those required for Ethereum block validation?*

Chapter 4

Solution Design

This chapter details the architectural and cryptographic design of the proposed lattice-based RISC-V ZkVM. It is structured to first establish the system's high-level architecture (4.1), describing the lifecycle of a proof from high-level Rust code to a finalized succinct argument. Following this, the specific cryptographic primitives are defined (4.2), selecting the LatticeFold folding scheme ([41]) as the core recursion engine, while identifying Greyhound [40] as the necessary wrapping SNARK for final verification. The design then proceeds to the construction of the RISC-V step circuit F (4.3), detailing how 32 bit integer arithmetic is arithmetized into the Goldilocks ring elements required by LatticeFold using CCS [44]. Finally, the recursive mechanism F' is defined (4.4), outlining how IVC [23] is enforced to prove the integrity of long running computations. The chapter concludes with the design of the final wrapping SNARK (4.5), which compresses the final folded artifacts into a succinct proof suitable for external verification.

4.1 High level Architecture

The proposed system architecture implements a post-quantum ZkVM capable of proving the correct execution of arbitrary RISC-V programs. The design leverages IVC to break down the execution of a program into individual steps. To instantiate IVC a folding scheme LatticeFold [41] is used, which recursively folds steps into a single, constant-sized cryptographic object. The architecture is designed to support the 32 bit RISC-V instruction set. Following recent zkVM systematization work [8], this design can be understood as a composition of an ISA layer, an execution layer that produces traces, and a proving layer that arithmetizes and proves them.

4.1.1 The ZkVM IVC

The lifecycle of a proof within this system proceeds through a pipeline of transformation, execution, and cryptographic accumulation, as illustrated in Figure 4.1. This workflow is defined by the following stages:

1. **Compilation and loading:** A program written in a high-level language (e.g., Rust) is compiled into a `rv32imac` ELF binary. This binary is loaded into the ZkVM emulator.

2. **Execution and trace generation:** The emulator executes the program one instruction at a time. For every step i, the system captures a comprehensive *execution trace*. This trace encapsulates the exact state transition of the machine. Mathematically, the trace for step i contains:

   - **Cycle count:** The step counter.

   - **Input state:** The program counter (PC) and the state of all 32 general-purpose registers before execution.

- **Output state:** The PC and register states after execution.
- **Decoded instruction:** The raw binary instruction broken down into its opcode, operands, and immediate values.
- **Side effects:** A set of flags indicating branching behavior (jump targets), and memory operations (address, value, and read/write flags).

3. **Arithmetization and witness generation:** The raw data are transformed into a vector of Goldilocks ring elements $\mathbb{Z}_q$. This vector serves as the *witness* for the constraint system. The logic verifying the transition from input state to output state based on the decoded instruction defines the **Step circuit**, denoted as F.

4. **The augmented circuit F':** To achieve IVC, the step circuit F is embedded inside a larger augmented circuit, F'. This circuit is responsible for the recursive logic. For a step i, F' performs the following operations:

   (a) **IVC integrity checks:** It recalculates the commitment to the previous step's data, ensuring the chain of computation is unbroken. This follows the IVC patterns established in Nova [29], SuperNova [24], and HyperNova [30].

   (b) **Folding verification:** It constrains the LatticeFold verifier. This enforces that the folding of step $i-1$ into the running accumulator was performed correctly.

   (c) **Step verification:** It enforces the constraints of F, guaranteeing that the RISC-V instruction at step i was executed correctly according to the ISA.

5. **Cryptographic folding:** The prover executes the non-interactive folding protocol off-circuit. It takes the current step's instance-witness pair (u_i, w_i) and the running accumulator from the previous step (U_{i-1}, W_{i-1}). Using the LatticeFold scheme, these are compressed into a new updated accumulator (U_i, W_i). This process generates a `folding proof`, which serves as a private input for the augmented circuit checks.

6. **Memory logging:** If the instruction execution triggers a memory operation (load or store), this side effect is captured and appended to a global vector of memory operations. Unlike register transitions, memory consistency is not fully proven within F', instead, it is verified via a deferred memory consistency check after the execution concludes [45].

7. **Final wrapping:** Once the program halts, the system holds a single folded instance-witness pair and final folding proof. These could be used to prove the execution, but they are not succinct. To produce a succinct proof suitable for blockchain verification, these large lattice-based artifacts are wrapped in a final SNARK as depicted in Figure 4.1. This wrapper consist of the SuperSpartan IOP [30] to prove the validity of the folded CCS instance, combined with the Greyhound [40] PCS to ensure post-quantum security and succinct verification.

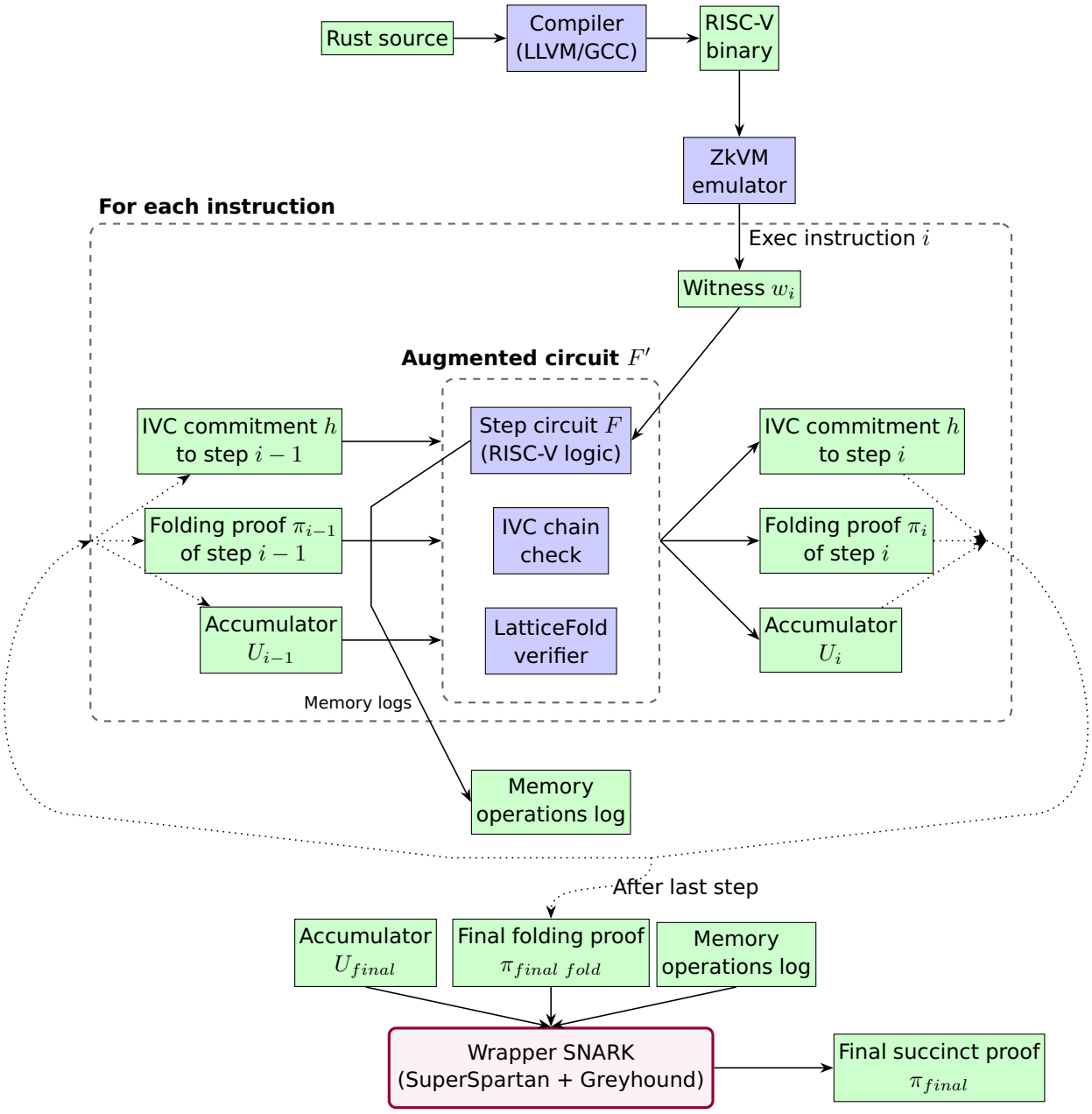

Figure 4.1: Execution flow of the lattice-based RISC-V ZkVM IVC instantiation.

4.1.2 Prover and Verifier Roles

The system distinguishes between the computational roles of the Prover and the Verifier, particularly regarding the recursive steps versus the final verification.

The Prover

The Prover is responsible for the entire computational workload of the VM. Its duties include:

- **Witness generation:** Running the RISC-V emulator to produce execution traces.
- **Folding:** Computing the non-interactive folding steps to compress the current step's witness into the running accumulator.
- **Proof generation:** Generating the final wrapping proof once the recursion is complete.

The Verifier

The verification logic exists at two distinct levels:

1. **The circuit verifier (in circuit):** This is the logic embedded within F' (the augmented circuit). It does not run on an external machine but is simulated by the Prover to generate the proof. It verifies the mathematical correctness of the folding mechanism at every step.
2. **The final verifier:** This is the external entity (e.g., an Ethereum smart contract). It receives the final proof. Its role is to supply initial public arguments and verify the succinct proof of the whole RISC-V VM execution.

4.2 Cryptographic Primitives

ZkVM integrates several zero-knowledge cryptography primitives to achieve IVC via folding. This compositional approach allows the system to benefit from the efficiency of folding in recursive steps while maintaining post-quantum security under lattice-based assumptions. The following section details the selected cryptographic building blocks. From the perspective of zkVM architecture, these choices belong mainly to the proving layer, which interacts closely with the ISA and execution design decisions surveyed in [8].

4.2.1 Incrementally Verifiable Computation (IVC)

The foundational theoretical model for this ZkVM is IVC, introduced by Valiant [23]. IVC enables a single proof to attest to the correct execution of a long-running computation $y = F^{(n)}(x)$ by recursively proving that the state at step i is valid and that a valid proof exists for step $i - 1$. RISC-V execution traces of unbounded length align with this model [24].

4.2.2 Folding schemes and Nova

To make IVC practical, this work utilizes the concept of *Folding schemes* as introduced in the Nova system [29]. Unlike traditional recursion, where a circuit must fully verify a SNARK at every step, folding allows the prover to compress two instances of the same relation (one representing the current step and one representing the accumulator) into a single instance. The cryptographic cost of folding is negligible compared to SNARK verification. The resulting recursive proof can then be compressed by a Spartan-style SNARK [46].

4.2.3 Customizable Constraint Systems (CCS)

CCS generalizes R1CS [44], allowing for high-degree gates without overhead. This is advantageous for representing IVC constraints (such as constraining Poseidon2's S-Box, which is exponentiation to the 7th degree [47, 48]). A CCS structure S consists of:

- Size bounds $m, n, N, \ell, t, q, d \in \mathbb{N}$ where $n > \ell$.
- A sequence of matrices $M_0, \ldots, M_{t-1} \in \mathbb{F}^{m \times n}$.
- A sequence of q multisets $[S_0, \ldots, S_{q-1}]$, where an element in each multiset is from the domain $\{0, \ldots, t-1\}$ and the cardinality of each multiset is at most d.
- A sequence of q constants $[c_0, \ldots, c_{q-1}]$, where each constant is from $\mathbb{F}$.

A CCS instance consists of a public input $x \in \mathbb{F}^{\ell}$. A CCS witness consists of a vector $w \in \mathbb{F}^{n-\ell-1}$. A CCS structure-instance tuple (S, x) is satisfied by a CCS witness w if:

$$\sum_{i=0}^{q-1} c_i \cdot \bigcirc_{j \in S_i} (M_j \cdot z) = \mathbf{0} \tag{4.1}$$

where $z = (w, 1, x) \in \mathbb{F}^n$, $M_j \cdot z$ denotes matrix-vector multiplication, $\bigcirc$ denotes the Hadamard (entry-wise) product between vectors, and $\mathbf{0}$ is an m-sized vector with entries equal to the additive identity in $\mathbb{F}$.

Informally, for each constant c_i, multiply it by the element-wise product of all $(M_j \cdot z)$ vectors where j comes from multiset S_i. Sum all these results together, and check if the final vector equals zero. If all constraints are satisfied, the result is zero, indicating the witness correctly follows the rules

defined by the system.

4.2.4 Lattice-based hardness assumptions

To achieve post-quantum security, the cryptographic primitives in this work rely on hardness assumptions over lattices rather than the Discrete Logarithm Problem (DLP) used in Nova or HyperNova. The security relies on the hardness of the Shortest Integer Solution (SIS) and Module-SIS problems. These problems remain hard even in the presence of quantum computers [34, 10, 11, 12].

4.2.5 LatticeFold

LatticeFold [41] can be thought of as a HyperNova [30], but with post-quantum guarantees. It utilizes the Module-SIS [38] problem to construct Ajtai vector commitment scheme. Unlike DLP-based folding, the challenge in LatticeFold is the growth of the witness norm during folding steps. To maintain the binding property of the commitments, the protocol incorporates a sum-check range proof to ensure the folded witness remains within a small, predefined norm bound. This allows the scheme to operate over small prime fields while providing $O(\log n)$ proof size and achieving transparency, as no trusted setup is required.

4.2.6 The wrapping SNARK SuperSpartan + Greyhound

While LatticeFold allows for efficient recursion, the final artifact is a large lattice-based accumulator that is expensive to verify directly. A wrapping SNARK is used to compress this into a succinct proof.

This wrapper is constructed by combining the **SuperSpartan** [46, 30] IOP,

which verifies the satisfiability of the CCS relation, with **Greyhound** [40], a lattice-based PCS with polylogarithmic verification complexity.

SuperSpartan is agnostic to the underlying algebraic structure (field vs. ring), allowing instantiation of the Sum-Check protocol [49] over the lattice ring R_q. The final polynomial evaluation query required by SuperSpartan is then proven using the Greyhound PCS, which supports the same ring structure [30, 40].

4.2.7 Memory consistency check

The correctness of the RISC-V execution depends on the integrity of the Random Access Memory (RAM). To avoid the massive overhead of embedding a Merkle Tree or cryptographic accumulator within the step circuit F, this work employs **Offline memory checking**, a technique formalized for SNARKs by Ozdemir, Laufer, and Boneh [45].

Instead of verifying every memory access immediately, the prover maintains a transcript tr of all memory operations (reads and writes) in the order they occurred. Each access is represented as a tuple $acc_i = (a_i, v_i, w_i)$ where $a_i \in \mathbb{F}$ is the address, $v_i \in \mathbb{F}$ is the value, and $w_i \in \{0, 1\}$ is a write-bit (1 for write, 0 for read).

To prove that these operations are consistent with read-over-write semantics, the system constructs a second transcript tr' by sorting tr lexicographically by (a, t), where each access is augmented with a time field $t_i = i$. The prover must demonstrate three properties:

1. tr' is a **permutation** of tr (same accesses, different order);
2. tr' is (a, t)**-ordered**: accesses are grouped by address and time-ordered

within each group;

3. tr' respects **read-over-write (RoW)** semantics: reads return the most recent written value to each address.

The full memory consistency proof consists of three components, as described by Ozdemir, Laufer, and Boneh:

1. **Permutation proof:** demonstrates that tr' is a permutation of tr. Both transcripts contain the same multiset of memory accesses.

2. (a, t)**-ordering proof**: verifies that tr' is correctly sorted by address and time within each address group, using conditional ordering and conditional uniqueness sub-proofs.

3. **RoW proof**: enforces read-over-write semantics, ensuring that each read returns the most recent written value to its address.

For the detailed construction and security analysis of these protocols, refer to the original work [45].

4.2.8 LatticeFold+

It is noted that a newer iteration of the folding scheme, LatticeFold+ [42], has been proposed. LatticeFold+ introduces algebraic range proofs and double commitments, which reduce the prover overhead by eliminating the need for bit decomposition during witness norm management. However, at the time of system design, the available libraries for LatticeFold+ did not support CCS constraints. Thus, this work utilizes the standard LatticeFold protocol.

4.3 Designing the RISC-V circuit (F)

This section details the construction of the step circuit F, which encodes the transition function of the RISC-V VM according to the RISC-V ISA [6, 7]. The design targets a 32 bit architecture.

4.3.1 Field representations and arithmetization

The target architecture is a 32 bit RISC-V machine. The state of this machine within the cryptographic proof system is represented by elements from the Goldilocks field. The Goldilocks prime is defined as $p = 2^{64} - 2^{32} + 1$. Since $p > 2^{32}$, a single field element can losslessly represent any 32 bit register value or memory address.

The LatticeFold protocol operates over cyclotomic rings. A Goldilocks field element is mapped into the ring R_q using a Number Theoretic Transform (NTT) representation, where the base field is a 3rd degree extension of the Goldilocks field, and the NTT form consists of 8 components.

Arithmetization is the process of converting numbers handled by the VM, captured in the execution trace, into cyclotomic ring elements over the Goldilocks field.

4.3.2 State representation

The execution state at any step i is encapsulated by the tuple z_i, which contains the following components:

- **Program counter (pc_i):** A 32 bit integer indicating the address of the next instruction.

- **Registers:** An array of 32 general purpose 32 bit registers.
- **Memory state:** Represented by the root of a Merkle tree covering the machine's RAM.
- **Memory log:** A commitment to the vector of memory operations performed up to step i.

The initial state z_0 serves as a public anchor for the computation. Its commitment z_{0_comm} binds the proof to a specific ELF binary (represented by a `code_comm` Merkle root), the entrypoint, and the initial zeroed state of memory and registers. All of these values are public, known to both prover and verifier, this means, they do not have to be recomputed or constrained by the prover within the circuit.

4.3.3 Instruction decoding and execution

For each step, the circuit F enforces valid transitions between z_{i-1} and z_i. This process involves the following constraints:

1. **Fetch:** The circuit verifies that the instruction executed corresponds to the value stored at pc_{i-1}. This is enforced by verifying a Merkle proof against the static `code_comm` root derived from the ELF binary.
2. **Validation:** To ensure that the claimed decoded instruction corresponds to exactly one valid instruction, the circuit enforces three conditions on the set of selector flags $S = \{z_{\texttt{IS_ADD}}, z_{\texttt{IS_MUL}}, \dots\}$:
   (a) **Booleanity (are 0 or 1):** $\forall s \in S, s \cdot (1 - s) = 0$.
   (b) **Mutual exclusion (only one is 1):** $\sum_{s \in S} s = 1$.

(c) **OPCODE consistency:** Each instruction word contains a 7 bit OPCODE[1]. To enforce consistency between the active selector flag and the instruction, opcode bits $z_{\texttt{INSTRUCTION}}[0:7]$ are constrained to match the weighted sum of opcode constants:

$$\sum_{s \in S} (s \cdot \texttt{OPCODE}_s) = z_{\texttt{INSTRUCTION}}[0:7] \tag{4.2}$$

Where $\texttt{OPCODE}_s$ is the corresponding constant to the instruction s[2]. Since only one flag s is active, this sum collapses to the constant $\texttt{OPCODE}_s$ of the selected instruction.

(d) **functX consistency:** Certain instructions need additional fields from their binary format to be uniquely decodable. Instruction flags subset of those that require funct3 is defined as $S_{\texttt{F3}} \subset S$ and subset for those that require funct7 as $S_{\texttt{F7}} \subset S$[3]. The circuit enforces:

$$\sum_{s \in S_{\texttt{F3}}} s \cdot (z_{\texttt{INSTRUCTION}}[12:14] - \texttt{F3}_s) = 0 \tag{4.3}$$

$$\sum_{s \in S_{\texttt{F7}}} s \cdot (z_{\texttt{INSTRUCTION}}[25:31] - \texttt{F7}_s) = 0 \tag{4.4}$$

Instructions that do not require these fields, are not in the subsets, and these terms equal to 0.

3. **Range checks:** Based on the decoded instruction, source registers ($rs1, rs2$), destination register (rd), and/or immediate values are checked inside the circuit. Register indices must be bounded $rs1, rs2, rd \in [0, 31]$ and immediate values are constrained to the valid bit width for each

[1] E.g. 0110011 for R type, register-to-register, instructions
[2] ADD and SUB are R type instruction, which means, for them $\texttt{OPCODE}_s = 0110011$
[3] Instruction ADD and SUB have the same OPCODE and funct3, but their funct7 differ

instruction type.

4. **Execution logic:** Based on the decoded instruction, the circuit enforces it's semantic logic.

5. **State update:** The circuit constrains the output state z_i. For arithmetic instructions, this involves updating the destination register. For branch instructions, it involves updating the pc.

4.3.4 Memory consistency checks

Memory consistency ensures that a load operation at step i returns the value written by the most recent store operation to that address. The verification of the memory consistency is deferred until the end, when all memory access logs have been collected. Done this way, the circuit inside IVC is less complex and does not need to keep track of the history of memory accesses in state.

When a load or store instruction is executed:

1. **Operation capture:** The circuit captures the operation, which contains the cycle count i, the address, the value, and a boolean flag indicating a write operation.

2. **Log update:** This operation is appended to the memory log, and new commitment (from the previous one and the parts of the new memory operation) is passed as public input. Within F, this new commitment is recalculated and compared to the supplied public input.

3. **Page validation:** For store operations, the circuit additionally verifies a Merkle proof for the memory page being modified, calculates the new page root, and updates the memory Merkle root in the state z_i.

Each memory event is constrained to match the executed instruction; only global read-over-write consistency is deferred. Instead, the correctness of the read/write history is verified in the final wrapping SNARK (discussed in subsection 4.2.6) using checks described in previous subsection 4.2.7.

4.4 IVC (F')

To achieve incrementally verifiable computation, the step circuit F is embedded within a larger augmented circuit, denoted as F'. This circuit is responsible for maintaining the integrity of the proof chain and verifying the correct folding of previous steps [29, 24].

4.4.1 Enforcing IVC integrity

The validity of the recursion relies on linking the current step i to the verified history of the computation terminating at step $i-1$. This linkage is enforced through the hash commitment chain. The augmented circuit F' accepts a public input h_{i-1}, which is a Poseidon2 [48] hash commitment to the state of the IVC scheme after the previous step. The private witness supplied by the prover contains preimage of the h_{i-1}. The preimage consists of:

1. **Previous step counter** $i-1$
2. **Initial state commitment** $z_{0_{\text{comm}}}$
3. **Previous state commitment** z_{i-1_comm}
4. **Previous accumulator commitment** U_{i-1_comm}.

Inside the circuit, the integrity check is performed by first recomputing the Poseidon2 step commitment h'_{i-1} from preimage elements from private wit-

ness. A constraint asserts that the recomputed candidate h'_{i-1} is equal to the provided public input h_{i-1}.

Secure all the way down

The IVC construction in Nova [29] enforces a security model with only a "one layered step commitment" h. However, in the context of a ZkVM, the step commitment h is not comprised of only raw values but of more cryptographic commitments.

If the internal structure of z_{i-1_comm}'s and U_{i-1_comm}'s preimages are not constrained within the circuit, a malicious prover could forge a h_{i-1} to commit to fake z^*_{i-1} or fake U^*_{i-1}.

To prevent such attacks and ensure the integrity of the virtual machine state transitions, the augmented circuit F' must recompute the Poseidon2 U_{i-1_comm} commitment from its preimage U_{i-1} supplied through the private witness. And also it must enforce constraints on the z_{i-1_comm}'s preimage:

1. **Code integrity:** The preimage contains the Merkle root of the program's binary code. This root enables the verification of instruction decoding via Merkle opening proofs. Since the program code is static and immutable, this root must not change during execution. To enforce this, the circuit accepts the expected code Merkle root as a public input (supplied by the verifier). The circuit constrains that the code root contained within the preimage of z_{i-1_comm} matches this global input, preventing the prover from executing instructions from a different program.

2. **Program counter:** The program entry point and current execution pointer are tracked. The circuit logic F determines the next pc based

on the current instruction (e.g., sequential execution or branching). The circuit constrains that the pc component in the preimage of z_{i_comm} matches this calculated value.

3. **Memory pages:** The VM memory is represented by a Merkle root. When a store instruction is executed, the circuit requires a Merkle opening proof for the target page against the root in z_{i-1_comm}'s preimage.

4. **Register state:** The state includes a Poseidon2 commitment to the 32 general purpose registers. The values of these registers before and after the instruction are provided as part of the private witness for the step circuit F. The circuit recalculates the commitment to the output register values and asserts equality with the register component of the z_{i_comm}'s preimage.

5. **Memory operations log:** The log represents a history of all memory operations. The commitment to this log must be updated incrementally. If a memory operation occurs during step i, the circuit computes the new log commitment by hashing the previous commitment (from z_{i-1_comm}) together with the details of the current operation. This result is constrained to match the log commitment in z_{i_comm}'s preimage.

4.4.2 The on circuit NIFS verifier

Following the integrity check, F' executes the verification logic for the LatticeFold Non Interactive Folding Scheme (NIFS). This verifies the mathematical correctness of the folding process without re-executing the prior computation.

The circuit accepts the folding proof π_{i-1} as part of the witness. It then executes the LatticeFold verifier with the previous accumulator U_{i-1} and proof π_{i-1}, both taken from the private witness. Upon successful verification of the folding, the circuit executes the RISC-V step circuit F, as detailed in Section 4.3.

4.4.3 Output generation

The execution of one IVC step concludes by generating the artifacts required for the subsequent step. This includes the new folded accumulator instance U_i, the new folding proof π_i, and the new IVC commitment h_i. h_i is calculated by hashing the updated step counter, the initial state commitment z_{0_comm}, the new state commitment z_{i_comm}, and the new accumulator commitment U_{i_comm} using Poseidon2. This h_i becomes the public input for the next iteration of the recursive loop.

4.4.4 IVC verification

After the final IVC chain terminates, the output is final witness-instance pair and a proof of last folding. These artifacts would be enough for proving that there exists a valid execution trace of a program P, staring from public initial state z_0, ending in public final state z_n, such that every instruction transition follows the selected RISC-V ISA, every memory operation is consistent with read-over-write semantics, and every folded accumulator is valid under LatticeFold. However, they are not succinct, so they must be wrapped in order to produce a succinct proof.

4.5 Final Wrapping SNARK

This section refines the high-level description from Subsection 4.2.6 into a concrete final proving layer. After the IVC chain terminates, the prover holds a final folded accumulator, its witness, the last folding proof, and the data required for deferred memory checking. These artifacts are sufficient to justify correctness of the execution, but they are not succinct enough for direct blockchain verification. The purpose of the final wrapper is therefore to compress them into a single succinct proof π_{final}.

4.5.1 Wrapped statement

The wrapper proves an NP statement that can be denoted by a relation $R_{wrap}(x_{pub}, w_{wrap}) = 1$. Its public input contains the program commitment, the initial state commitment z_{0_comm}, the claimed final state commitment z_{n_comm}, and the claimed public result of the computation. The private witness contains:

- the final accumulator instance-witness pair (U_n, W_n),
- the final folding proof π_{n_fold},
- the auxiliary witness needed to re-check the last IVC transition,
- the deferred memory witnesses, including the original transcript tr and its sorted form tr'.

The relation R_{wrap} is satisfied if and only if all of the following hold:

1. the final folded CCS accumulator is valid,
2. the last folding proof is valid and consistent with that accumulator,

3. the claimed final state commitment matches the state obtained after the last VM transition,

4. the memory transcripts satisfy permutation, (a, t)-ordering, and read-over-write semantics.

4.5.2 SuperSpartan layer

To prove satisfiability of R_{wrap}, the proposed design uses a SuperSpartan-style IOP [46, 30, 44]. This is a natural fit, because the internal objects of the zkVM are already expressed as CCS relations. Instead of embedding yet another full SNARK verifier into the wrapper circuit, SuperSpartan reduces the final CCS statement to a small number of sum-check and polynomial evaluation claims.

The witness of the wrapper contains the same algebraic objects that appear at the end of the IVC process: accumulator, folding witness variables, and memory-check witnesses. SuperSpartan verifies these through a sum-check based reduction, leaving only polynomial evaluation queries to be proven succinctly.

4.5.3 Greyhound layer

The remaining polynomial evaluation claims are proven with Greyhound [40]. This choice is important for two reasons. First, Greyhound is a lattice-based polynomial commitment scheme, so it preserves the post-quantum assumption stack of the overall zkVM. Second, it provides succinct evaluation proofs with sublinear verification complexity, which is exactly the final compression step needed after SuperSpartan.

The final proof remains in the same general lattice-based design space as the folding layer itself. The Greyhound paper reports evaluation proofs around 53 KB for large polynomial degrees, which suggests that a future fully implemented could plausibly produce a final proof in the range of tens of kilobytes, rather than exposing the much larger native accumulator.

4.5.4 Soundness

The soundness of the final proof is obtained by composition. If π_{final} verifies, then the soundness of the SuperSpartan and Greyhound wrapper implies that there exists a witness satisfying R_{wrap}. By construction of R_{wrap}, this witness contains a valid final folded accumulator and a valid final folding proof. By soundness of the LatticeFold recursion, this implies the existence of a consistent unfolded IVC chain. By the step circuit design from Section 4.3, each step in that chain corresponds to a valid RISC-V transition. Finally, the deferred memory proof ensures that all logged memory accesses satisfy read-over-write semantics.

Therefore, an accepting final wrapper proof implies existence of a valid execution trace of the committed program, starting from the public initial state and ending in the claimed final state.

Chapter 5

Implementation

This chapter presents the prototype implementation of the lattice-based RISC-V ZkVM architecture described in the Chapter 4. The implementation covers:

- A subset of rv32imac instructions sufficient to run Nth Fibonacci and validate an Ethereum block with REVM.
- CCS constraints for these instructions.
- CCS constraints for the Poseidon2 IVC step commitment.
- CCS constraints for NIFS verifier.

Regarding the choice of folding scheme, this work uses the NetherMind's implementation of LatticeFold protocol rather than its successor LatticeFold+, as the latter's library implementation was not available at the time of system design and development.

5.1 Technology stack

The prototype implementation of the lattice-based RISC-V ZkVM is developed using the Rust programming language (`nightly-2025-08-19`). Rust was selected due to its dominance in the cryptography and zero-knowledge ecosystem.

The cryptographic foundations and virtual machine components rely on several crates:

- **Plonky3**[1]**:** The `Plonky3` meta crate is utilized to provide the core arithmetization primitives. The `p3-field` and `p3-goldilocks` crates are used for Goldilocks field arithmetic. `p3-poseidon2` for the construction of Poseidon2 hashes and `p3-merkle-tree` for memory and code commitments.
- **LatticeFold**[2]**:** The core folding mechanism is based on the `latticefold` implementation provided by Nethermind. This crate implements the Ajtai commitment scheme and the NIFS prover and verifier logic. It is the primary component of the IVC loop, enabling the recursive accumulation of execution steps.
- **Cyclotomic-rings**[2]**:** From the same source as LatticeFold crate, the `cyclotomic-rings` crate is used to handle the specific algebraic structures required for lattice-based cryptography over ring $R_q = \mathbb{Z}_q[X]/(X^d + 1)$. It provides the necessary NTT implementations and ring arithmetic for polynomials over the Goldilocks field.
- **REVM**[3]**:** Used as a guest program in the ZkVM which validates an

[1] https://github.com/Plonky3/Plonky3
[2] https://github.com/NethermindEth/latticefold
[3] https://github.com/bluealloy/revm

Ethereum block.

5.2 Cryptography

Cryptographic crates (and majority of all cryptographic libraries, not just for Rust) expose high level API. Developers provide inputs to this API and get back the desired hash, encrypted text, key, etc. In context of ZkVM, also the intermediate states are needed to populate the witness. Main hurdle during implementation was to reuse exposed parts of used cryptographic crates, in order to collect internal states of these primitives.

5.2.1 Poseidon2

The Poseidon2 hash function is used to commit to the IVC state. While the `Plonky3` library provides a implementation of Poseidon2, its standard interface treats the permutation as a black box, returning only the final digest. To correctly constrain the Poseidon2 execution within a CCS requires access to the intermediate states of the permutation. This is because Poseidon2 has a 7th degree S-box, which is dependent on previous rounds, and those on ones before them. Thus, the computation is broken down into smaller steps, where each step verifies a transition of a one computational segment.

A custom implementation of the Poseidon2 permutation was constructed, structurally identical to the `Plonky3` implementation but modified to capture and return the state vector at desired boundaries:

- The state after the initial MDS matrix application.
- The state after each of the external initial full rounds.

- The state after each of the internal partial rounds.
- The state after each of the external terminal full rounds.

5.2.2 LatticeFold

In practice, integrating LatticeFold into the ZkVM required vendoring the `latticefold` repository and using it as a local workspace dependency instead of consuming it purely as an upstream dependency. The main reason was that the ZkVM needs access not only to the final prover and verifier outputs, but also to internal values that are useful for witness generation and for constructing CCS constraints for the in-circuit verifier. Some helper functions and verifier-side routines in the original library were private, which made them unavailable for reuse from the ZkVM crate. Therefore, selected pieces of functionality had to be made public, and in a few places thin wrapper functions were added in order to reuse the original implementation rather than duplicating the protocol logic.

5.3 RISC-V Emulator

A custom, lightweight RISC-V interpreter was developed to execute the program and generate the necessary cryptographic witnesses. The emulator supports the `rv32imac` instruction set architecture. Since it being only a PoC, not all instruction handlers are implemented, but sufficient amount of them to execute Rust binaries compiled for the `riscv32imac-unknown-none-elf` target.

The core of the emulator is structured around an execution loop that fetches, decodes, and executes instructions. At each instruction step, the execution

halts and the full machine state, including the program counter, registers, memory pages, and raw instruction bytes, is exposed to an interceptor.

This way, the instruction semantics and the proof generation are decoupled. The emulator is responsible solely for correct state transitions and the generation of the execution trace, while the ZkVM utilizes this trace to generate witnesses.

5.4 Circuit synthesis

The constraints defining the validity of the VM execution and the IVC chain are synthesized into a CCS.

5.4.1 Poseidon2 constraint example

This section illustrates the construction of CCS constraints for Poseidon2 hash [48]. Poseidon2 permutation in Goldilocks field on 12 elements (rate is 12, with capacity 4, total width then being 16) consists of:

1. Applying a maximum distance separable MDS matrix.
2. 4 external initial rounds a 7th degree S-box on all elements.
3. 22 internal rounds with a partial S-box applied only to the first element.
4. 4 external terminal rounds a 7th degree S-box on all elements.

Initial MDS application. The first operation applies the MDS matrix to the absorbed state. The input 12 elements and 4 zeroes must be in the witness. Here this input will be denoted as $s = (s_0, s_1, \ldots, s_{11}, 0, 0, 0, 0)$. Then the output of the MDS application is captured into the witness, denoted as

s_{after_mds}. The constraint enforces

$$s_{after_mds_i} - s'_i = 0 \tag{5.1}$$

where $i \in (0, 1, \ldots, 15)$ and $s' = \texttt{MDS} \cdot s$.

External initial rounds. Next are external rounds where a round constant is added and S-box is applied to all state elements. There are 4 external initial rounds, after each the state of 16 elements must be captured (denoted as $s_{\text{ext_init_r_i}}$; $r \in (0, 1, 2, 3)$; $i \in (0, 1, \ldots, 15)$). Then in circuit, for each round there is a constraint:

$$\texttt{MDS}^{-1} \cdot s_{\text{ext_init_r_i}} - (s_{\text{in_r_i}} + c_{\text{ext_init_r_i}})^7 = 0 \tag{5.2}$$

where $c_{\text{ext_init_r_i}}$ is the specific constant at index i for round r, and the $s_{\text{in_r_i}}$ is the input to the round, in first one, it is the s_{after_mds}, otherwise it is the output of previous round.

Internal rounds. Following the external initial rounds, the permutation executes 22 partial internal rounds where the S-box is applied only to the first state element. The state after each round is captured into witness, denoted as $s_{\text{inter_r_i}}$; $r \in (0, 1, \ldots, 21)$; $i \in (0, 1, \ldots, 15)$. Each internal round computes for each first element:

$$M_I^{-1} \cdot s_{\text{inter_r_0}} - (s_{\text{in_r_0}} + c_{\text{inter_0}})^7 = 0 \tag{5.3}$$

For other than first element ($i \in (1, 2, \ldots, 15)$ it does:

$$M_I^{-1} \cdot s_{\text{inter_r_i}} - s_{\text{in_r_i}} = 0 \tag{5.4}$$

The $s_{\text{in_0}}$ is the output of the last external initial round, other $s_{\text{in_i}}$ are outputs of the previous internal rounds.

External terminal rounds. The permutation concludes with 4 external terminal rounds, mirroring the external initial rounds but using terminal round constants and different inputs $s_{\text{ext_term_r_i}}; \quad r \in (0, 1, 2, 3); \quad i \in (0, 1, \ldots, 15)$):

$$\texttt{MDS}^{-1} \cdot s_{\text{ext_term_r_i}} - (s_{\text{in_r_i}} + c_{\text{ext_term_r_i}})^7 = 0 \tag{5.5}$$

where $c_{\text{ext_term_r_i}}$ is the specific constant at index i for round r, and the $s_{\text{in_r_i}}$ is the input to the round, in first one, it is the last internal round, otherwise it is the output of previous external terminal round.

Output extraction. After the final external terminal round, the output digest is extracted from first 4 elements of the state

$$h = (s_0, s_1, s_2, s_3) \tag{5.6}$$

And compared to the claimed result.

Complexity analysis. For a single Poseidon2 sponge pass, the constraint system captures 388 witness variables across the permutation computation. The system requires 44 matrices to encode the transformations, round constants, and S-box operations. The number of matrices remains constant regardless of the number of sponge passes, though their dimensions scale with

the permutation state size. The highest degree constraint is a 7th degree multiset constraint corresponding to the Goldilocks S-box, and this degree does not increase with additional sponge passes.

5.4.2 Self-aware NIFS constraints

Some constraints needed for the in-circuit NIFS verifier are not local in the sense that they do not only depend on a fixed small set of witness variables, but on the final structure of the whole CCS. The most notable example is the linearization inner-product check, which verifies the relation

$$\texttt{inner} = \sum_i c_i \cdot \prod_{j \in S_i} u_j, \tag{5.7}$$

where the multisets S_i are precisely the multisets of the CCS currently being built. Therefore, this part of the verifier is self-aware. It must inspect the complete list of already created multisets and generate constraints from them.

In the implementation this means that such constraints cannot be synthesized in a single pass. The builder first creates all ordinary VM, Poseidon2, decomposition, and folding constraints. Only after the full multiset structure is known are the matrices for the linearization inner-product check preallocated. Afterwards, in a second pass, these preallocated matrices are filled row-by-row with the concrete indices corresponding to the final CCS multisets. This is the reason why the folding related part of the builder must ordered carefully, and why the constants describing the final CCS dimensions (such as the number of matrices, multisets, and $\log_2(m)$) must be kept in sync with the generated system.

5.5 Implementation optimizations

Besides the direct translation of algebraic relations into CCS constraints, two optimizations were done in order to keep the ZkVM runnable on local consumer-grade hardware as long as possible in development.

5.5.1 `arkworks-rs` upgrade

The LatticeFold library, and its dependencies, use the `arkworks-rs`[4] version `v0.4.x` libraries for low level cryptography primitives. New versions of these foundational libraries were released with optimizations and improvements. In order to update and gain these benefits, the local fork of `latticefold` was migrated to new version along with forking its dependencies `cyclotomic-rings`, `stark-rings` and also migrating them to new versions.

5.5.2 Reusing matrix structures

Another optimization is that the builder groups together constraints that share the same algebraic shape, even if they correspond to different parts of the verifier. Instead of creating completely separate matrices and multisets for every single row, the implementation often allocates one small family of matrices and reuses them across many rows. For example, all product constraints of the form "left input times right input equals intermediate product" can be stored in the same pair of matrices and one linear matrix for the output product. Likewise, entire families of recomposition constraints are accumulated into shared linear matrices, differing only in the rows that are populated. This reduces CCS size and avoids unnecessary growth in the

[4] https://github.com/arkworks-rs

number of matrices and multisets.

5.6 High level overview

Figure 5.1 illustrates the complete execution flow. The setup phase prepares the VM by loading the ELF binary and constructing the CCS constraint system. Within the execution loop, four main modules work together: the `riscvm` module handles instruction execution and trace generation, the `commitments` module creates Poseidon2 hashes for state components, the `ivc` module arithmetizes the trace into the CCS witness vector, and the `main` module performs the folding operations that compress each step into the accumulator. The dashed line on the right shows the recursive nature of the computation, where the `IVCStepOutput` from step i becomes the `IVCStepInput` for step $i + 1$.

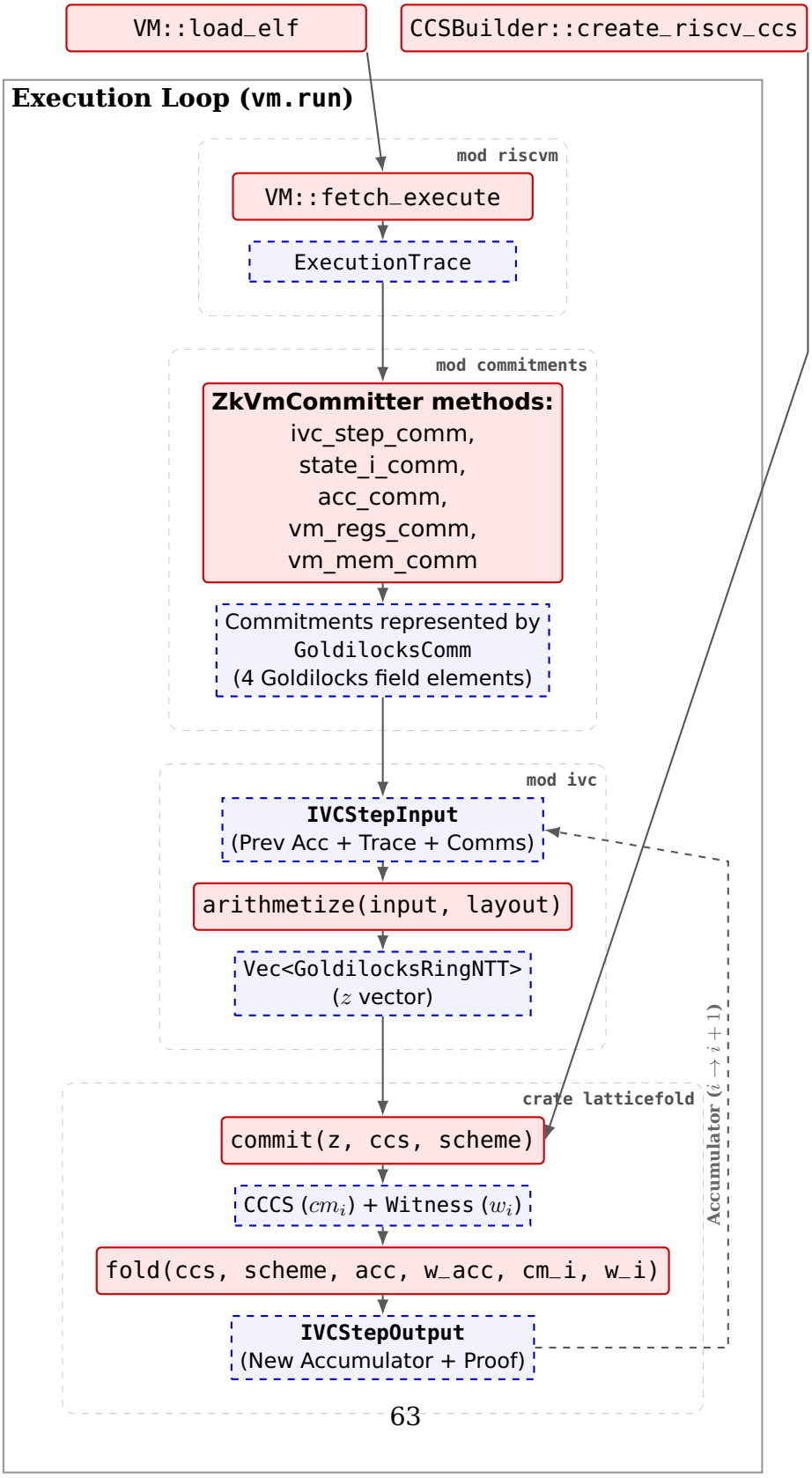

Figure 5.1: High-level overview of the implemented execution flow.

5.6.1 Toolchain summary

The implementation is built with the following software stack:

- **Rust:** `nightly-2025-08-19`, edition 2024.
- **Targets:** native host execution for the prover and `riscv32imac-unknown-none-elf` for guest programs.
- **LatticeFold:** a fork of Nethermind's `latticefold` implementation, used for Ajtai commitments, decomposition, linearization, and NIFS folding.
- **Plonky3 core crates:** `p3-field`, `p3-goldilocks`, `p3-poseidon2`, `p3-merkle-tree`, `p3-commit`, `p3-matrix`, `p3-mds`, `p3-challenger`, and `p3-symmetric`.
- **REVM:** version `36.0.0`, used inside the `evm` guest for replaying Ethereum block execution.

5.6.2 Instruction coverage

Emulator doesn't have a full `rv32imac` instruction coverage, just enough for running a Fibonacci program and proving a block with REVM. And also the zkVM doesn't constraint all the supported instructions.

Implemented in the VM emulator:

- **RV32I control flow:** `LUI`, `AUIPC`, `JAL`, `JALR`, `BEQ`, `BNE`, `BLT`, `BGE`, `BLTU`, `BGEU`.
- **RV32I memory:** `LB`, `LH`, `LW`, `LBU`, `LHU`, `SB`, `SH`, `SW`.
- **RV32I integer arithmetic and logic:** `ADDI`, `SLTI`, `SLTIU`, `XORI`, `ORI`, `ANDI`, `SLLI`, `SRLI`, `SRAI`, `ADD`, `SUB`, `SLL`, `SLT`, `SLTU`, `XOR`, `SRL`, `SRA`, `OR`, `AND`.
- **RV32M:** `MUL`, `MULHU`, `DIVU`, `REMU`.

- **RV32A:** `LR.W`, `SC.W`, `AMOADD.W`.
- **Other:** `FENCE`, `ECALL`. The local compressed decoder also contains an explicit fallback for `C.SUB`, `C.XOR`, `C.OR`, and `C.AND`, which are translated to their canonical RV32I forms.

Unsupported:

- **RV32I:** `EBREAK`.
- **RV32M:** `MULH`, `MULHSU`, `DIV`, `REM`.
- **RV32A:** `AMOSWAP.W`, `AMOXOR.W`, `AMOAND.W`, `AMOOR.W`.
- **RV32A:** `AMOMIN.W`, `AMOMAX.W`, `AMOMINU.W`, `AMOMAXU.W`.

Constrained inside the zkVM:

- `LUI`, `AUIPC`, `JAL`, `JALR`, `BNE`, `SW`, `ADDI`, and `ADD`.

5.6.3 Parameter set

The implementation fixes the following algebraic and cryptographic parameters:

- **Base field:** the Goldilocks prime field with modulus $p = 2^{64} - 2^{32} + 1 = 18446744069414584321$.
- **Cyclotomic ring:** coefficient representation over $R_q = \mathbb{F}_q[X]/(X^{24} - X^{12} + 1)$, so the ring degree is 24.
- **CRT/NTT form:** the ring splits into 8 NTT components over $\mathbb{F}_{q^3}$, i.e. extension degree 3 and `TAU` $= 24/8 = 3$.

- **Challenge set:** short lattice challenges use coefficients in the range $[-32, 32)$.
- **Ajtai/LatticeFold decomposition parameters:** $B = 2^{15}$, $L = 5$, $B_{\text{small}} = 2$, and $K = 15$.
- **Commitment module rank:** $\kappa = 32$ ring elements.
- **Poseidon2 commitments:** output width 4, narrow state width/rate $(8, 4)$, wide state width/rate $(16, 12)$, full rounds 8, partial rounds 22, and S-box degree 7.
- **Hash security estimate:** the wide Poseidon2 sponge uses capacity 4 over a 64-bit field, which gives roughly 128-bit security.
- **Commitment matrix width:** the Ajtai commitment uses $N = \texttt{w_size} \cdot L = 19763 \cdot 5 = 98815$ columns.

5.6.4 CCS dimensions

The resulting instance has:

- witness layout size $\texttt{w_size} = 19763$,
- total z-vector size $n = 19768$,
- public input size $\ell = 4$,
- padded row count $m = 131072 = 2^{17}$,
- number of matrices $t = 125$,
- number of multisets $q = 52$,
- maximal multiset degree $d = 7$,

- row logarithm $s = 17$,
- column logarithm $s' = 15$.

Chapter 6

Evaluation

This chapter evaluates the lattice-based RISC-V ZkVM prototype developed in this work. The current implementation is incomplete and should not be understood as a production-ready ZkVM: the software VM supports a larger instruction set than the subset currently constrained inside the zkVM, the final Greyhound-based wrapping proof is not yet integrated, and the full recursive workflow is not yet proved end-to-end inside in-circuit. The goal of this chapter is therefore not to claim state-of-the-art performance, but to compare the prototype with existing ZkVMs, to explain the observed differences, and to answer the research questions of this thesis.

The performance measurements were conducted on server-grade hardware with the following configuration:

- **CPU:** Intel Xeon E312xx
- **OS:** Ubuntu 24.04
- **Memory:** 64 GB RAM

6.1 Fibonacci benchmark

A benchmark was prepared with two `fibonacci` guest variants. The first, non-optimized variant computes the 100th Fibonacci number inside the guest itself. Its compiled binary contains around 60 instructions. That full runtime trace is too long to prove and benchmark comfortably in the current prototype. For the reported measurements, the optimized guest variant was therefore used instead, where the compiler effectively materializes the wrapped 32-bit 100th Fibonacci value as a constant and the program only writes it to memory. This, not a Fibonacci calculation, but nevertheless enough of a program, keeps the benchmark at a reasonable scale while still exercising the proof pipeline.

This simple benchmark program execution in the prototype resulted in:

- **Execution traces:** 16 instructions
- **Memory operations:** 2 operations
- **Total folding time:** 688.24 seconds

This result already shows that the current implementation is considerably slower than modern ZkVM systems. However, this number must be interpreted carefully, because the prototype still contains many unoptimized components and runs only on CPU.

6.2 Comparison with other ZkVMs

Additional benchmark artifacts were collected on the same machine for Pico, SP1, ZKsync Airbender, and Nexus zkVM 3.0. Table 6.1 compares the wall-clock time of the proving and the peak RAM usage.

ZisK is not included in this comparison. The benchmark server did not support the SIMD instruction set required by ZisK, and therefore a comparable proof run could not be executed on the same hardware.

ZkVM	**Proof time**	**Peak RAM**
This work	11:28.92	12.34 GB
Pico	2:04.50	6.27 GB
SP1	1:50.17	9.96 GB
ZKsync Airbender	4:06.73	24.34 GB
Nexus zkVM 3.0*	0:22.81	0.12 GB

Table 6.1: Comparison of proof generation time and peak RAM usage for the benchmark program.

** The Nexus zkVM 3.0 result reflects the documented proof flow from the public SDK quick-start. The public Nexus documentation consulted for this thesis does not provide a concrete instructions for the full recursive/wrapped proving pipeline, so that pipeline was not run.*

The comparison shows that the proposed system is slower than the compared systems. Pico and SP1 complete the proof much faster, Nexus zkVM 3.0 is also substantially faster, and Airbender is also significantly faster in time, although it uses substantially more memory. As recent systematization work emphasizes, such differences are influenced not only by the proof system itself, but also by ISA choice, trace layout, memory checking strategy, and recursion architecture [8].

At the same time, one possible advantage of the lattice-based construction may be proof size. The current prototype does not yet produce the final Greyhound wrapping proof, so no measured proof size can be reported at this stage. Therefore, the following comparison is only theoretical.

For the compared ZkVMs, the reported final proof sizes are approximately 1

MB for Airbender, approximately 1 MB for Pico, and approximately 260 KB for ZisK, as summarized in the earlier ZkEVM comparison in this thesis. In contrast, the Greyhound paper reports polynomial evaluation proofs of about 53 KB for large polynomials of degree up to $N = 2^{30}$. This is not directly the same as a final wrapping proof for the proposed ZkVM, so these values are not fully comparable. However, they indicate that a future Greyhound-based wrapper could potentially produce proofs in the order of tens of kilobytes. If this can be achieved in a complete implementation, proof size could become an important practical advantage of the lattice-based approach.

For the benchmark, each ZkVM was run according to its own official setup guide. This thesis followed the instructions from the setup guide in this document's appendix. Pico followed its quick-start guide[1], SP1 followed the official quickstart[2], ZKsync Airbender followed its guide[3], and Nexus zkVM 3.0 followed its SDK quick-start[4]. To keep the comparison practical and reproducible, all systems were exercised on the same optimized `fibonacci` guest.

To complement the small benchmark, a longer experiment was also run with the `evm` guest based on `REVM`. The guest program proved a Etherum block 24740084. This run continued for approximately 9 hours and 29 minutes on the same machine. During this time, the system kept processing the execution and verifying the IVC chain, while peak RAM usage remained around 13.7 GB. Unfortunately, the run didn't complete, because of a bug in the implementation, where the zkVM panicked due to incorrect logic of a instruction handler. This is far from production-level performance, but it is an

[1] https://pico-docs.brevis.network/getting-started/quick-start.html
[2] https://docs.succinct.xyz/docs/sp1/getting-started/quickstart
[3] https://github.com/matter-labs/zksync-airbender/blob/main/docs/overview.md
[4] https://docs.nexus.xyz/zkvm/development/sdk-quick-start

encouraging sign that the implementation can continue working on a substantially more complex workload without an uncontrolled memory increase or failing to fold inside the IVC chain.

6.3 Why is the prototype slower?

The benchmark results suggest that the main limitation of the current prototype is not correctness, but performance. Several reasons can explain this outcome.

Folding is not as cheap in practice as it appears in theory. The literature often describes folding as a sequence of relatively cheap random linear combinations. From a theoretical perspective this description is correct. However, the practical implementation cost is still high.

In the prototype, a significant amount of time is spent in multilinear extension-related computations. Even if the algebraic step is conceptually only a random linear combination, the real implementation still has to evaluate large objects, move large vectors through memory, and repeatedly perform arithmetic on high-dimensional representations. Therefore, even though LatticeFold is efficient in theory, it is not automatically efficient in practice.

This observation is supported by profiling of the benchmark proving run. The flame graph (which can be found in the project source code named `fibonacci_flamegraph.svg`) shows that most of the runtime is spent inside Rayon worker threads, where the dominant application hotspots belong to `latticefold`'s multilinear extension and folding/decomposition routines. The most latticefold related functions are `compute_mz_mles`, `to_mles_err`, `evaluate_mles`, `get_etas`, `compute_u_s` and there is also visible time spent

in `DenseMultilinearExtension` arithmetic.

By contrast, the commitment computation is present in the profile but is not one of the main bottlenecks, and witness arithmetization itself is not especially prominent. The profile therefore suggests that the main practical cost of the current prototype is not the front-end VM execution, but the MLE-heavy algebraic work required by decomposition and folding. This is consistent with the measured runtime, where the proving overhead is dominated by the algebraic representation of the witness rather than by the small benchmark program itself.

CCS may be an inconvenient computation model for this use case. Another possible reason is the choice of CCS as the computation model.

CCS is flexible and is well suited for expressing linear relations. This makes it a reasonable model for several simple VM instructions. However, the full system does not contain only linear computation. The non-interactive folding scheme itself is not purely linear, and the verifier logic introduces higher degree structure.

This creates several problems in practice:

- CCS does not provide custom gates in the same sense as many modern proving systems. Because of that, the representation may require more matrices and more bookkeeping than a more specialized system.
- Some parts of the VM are relatively simple in CCS, but components such as Poseidon2 and the folding verifier are much less natural to express.
- The folding verifier is especially difficult because its constraints must

be self-aware. In other words, they must already know the shape of the proof that they are verifying.

This last point may create a kind of circular dependency. To verify an object of higher degree D, the verifier may require information whose own constraint system increases the degree again. Then one has to enlarge the representation, which may increase the maximal degree once more. In this sense, the model may create a loop where expressing the verifier makes the verifier itself more expensive to express.

This also suggests that lookup arguments may be a useful complement to the current design. Lookup systems such as Lasso [50] can encode repeated table relations more compactly than direct constraint expansion, which may be relevant for operations on bytes, value checks, and other common VM subroutines.

For this reason, CCS may be acceptable for basic linear VM instructions, but it may be a poor fit for cryptographic permutations such as Poseidon2 and for the self-referential structure of folding verification.

6.4 Potential speed improvements

The current benchmark should not be understood as a final performance limit. Several optimization directions remain open.

6.4.1 SIMD and GPU acceleration

The present implementation runs only on CPU. This immediately puts it at a disadvantage compared to systems that use heavily optimized parallel proving pipelines.

The arithmetic in the prototype has regular structure, which makes it suitable for SIMD acceleration. Large vector operations, MLE evaluations, and some parts of the commitment computations could benefit from parallel execution. GPU acceleration is another natural direction, especially for large batches of similar field or ring operations.

There is also some encouraging ecosystem progress in this direction. Recently, a pull request was merged into the Arkworks algebra libraries whose primary motivation was to create a generalized path toward vectorized SIMD optimizations for finite fields in Arkworks[5]. This is relevant, because the current implementation depends on Arkworks-based components. However, using this new path in practice would still require non-trivial work. In particular, the `latticefold` library would need to rewrite important parts of its internal representation to use the new SIMD-friendly field types. Therefore, this optimization direction is promising, but it is not something that can be enabled by a small configuration change.

6.4.2 More mathematical optimizations

Because the system is built over algebraic structures, it may admit more specialized optimizations than a direct implementation currently uses.

Possible examples include:

- more efficient layouts for multilinear extensions,
- better batching of linear algebra operations,
- reuse of intermediate evaluations across folding steps,
- specialization of constraints for the most common instruction classes.

[5] https://github.com/arkworks-rs/algebra/pull/1044.

These improvements are not only low-level engineering optimizations. They are also mathematical optimizations, because the efficiency of the system depends strongly on how the algebraic objects are represented and evaluated.

6.5 Addressing the research questions

Feasibility of lattice-based instantiation

Is it feasible to instantiate a full RISC-V execution environment using CCS compatible with the LatticeFold protocol?

Answer: Partially yes.

From the design perspective, the thesis demonstrates that such an architecture can be specified. The mapping from RISC-V execution state to CCS-compatible constraints was designed, and the connection to LatticeFold was established.

From the implementation perspective, the feasibility is only partially shown. The prototype contains a functional RISC-V emulator, witness generation, and the basic integration with folding. However, it still lacks complete instruction set coverage and a complete in-circuit verifier for the full recursive workflow.

Therefore, the answer is positive on the architectural level, but still incomplete on the implementation level.

Performance trade-offs

How does the performance of a lattice-based folding scheme compare to established approaches of proving a general-purpose VM execution?

Answer: In the current prototype, the lattice-based approach is slower.

The benchmark comparison shows that the implemented system needs more time than the compared ZkVMs to generate a proof for the benchmark program. The main practical reason appears to be that the algebraic work of folding, especially MLE-related computation, is expensive in the current implementation. Another reason is that CCS may not be the most convenient model for all parts of the system, especially for cryptographic permutations and folding verification.

At the same time, the results do not imply that lattice-based folding is a bad direction in principle. The implementation is still early, CPU-only, and not yet optimized. In addition, compact proof size remains a positive property of the approach.

Therefore, the present answer is that the current implementation has worse practical proving time, but the long-term trade-off remains open.

Suitability for long-running computations

Does the noise growth inherent in lattice-based cryptography hinder the ability to prove long execution traces, such as those required for Ethereum block validation?

Answer: Yes, the results are encouraging.

From the design perspective, LatticeFold controls witness growth through its

decomposition layer, so the witness norm is kept within the protocol bounds even as the execution chain grows. That means long computations are not expected to fail simply because the witness becomes too large.

From the implementation perspective, the long-running `REVM`-based EVM experiment, although that run eventually stopped because of a bug in an instruction handler, it kept progressing for more than 9 hours and the IVC chain continued to fold successfully. This is a positive sign that the witness norm remained low enough for LatticeFold to keep operating. If the norm had grown beyond the allowed bound, the decomposition and folding checks would have failed and the IVC chain would not have been able to continue.

The prototype does not yet complete the full long-running EVM experiment, but the observed behavior shows that latticefold-based norm control is working and that long executions are not being blocked by witness blow-up.

Chapter 7

Conclusion

This thesis investigated whether a general-purpose RISC-V ZkVM can be built from lattice-based cryptographic primitives and still remain compatible with an IVC-style proving architecture. The main result is a concrete design for such a system. The proposed architecture combines a 32-bit RISC-V execution model, CCS-based arithmetization, LatticeFold recursion, deferred memory checking, and a Greyhound wrapping SNARK proof. In this sense, the thesis provides not only a conceptual motivation for a post-quantum ZkVM, but also a blueprint of how the individual components can be connected into a single proving pipeline.

The implementation demonstrates that the main building blocks of this design can be made to work together. A functional RISC-V emulator was implemented, witness generation for the execution trace was integrated with CCS construction, Poseidon2 was constrained inside the proving system, and the LatticeFold IVC chain was connected to the execution layer. The experimental results show that the current prototype is significantly slower than established zkVMs, and profiling indicates that the dominant cost lies in

multilinear-extension and folding related algebraic computation rather than in the VM execution itself.

At the same time, the results are not purely negative. The system successfully produced artifacts for the benchmarked execution, and a longer `REVM`-based EVM experiment continued for more than nine hours while the IVC chain was being verified and memory usage remained bounded. This does not yet show practical competitiveness with production zkVMs, but it does support the central claim of the thesis: a lattice-based, non-hash-based, post-quantum ZkVM is feasible at the architectural level, and its current limitations appears to be mainly engineering and arithmetization issues rather than a fundamental impossibility of the approach. In addition, although the prototype does not yet implement the final Greyhound wrapper, the expected proof size of such a wrapper remains attractive. Based on Greyhound's reported evaluation proof sizes, a future final proof in the range of a few tens of kilobytes appears plausible, which would be a strong practical property of this design.

7.1 Future Work

Several directions for future research and development emerge from this work:

- **Different computation model than CCS:** One important question is whether CCS is the right arithmetization for a self-proving IVC system. While CCS is expressive and convenient for some VM instructions, the evaluation suggests that it may be a poor fit for recursive verifier logic and for some non-linear cryptographic components. Alternative models, such as AIR or lookup formulations, may be a better foundation for

future iterations.

- **SIMD and GPU acceleration for lattice operations:** A large part of the current prover cost comes from lattice arithmetic, decomposition, multilinear extensions, and related linear algebra. These operations are regular and parallel, which makes them promising candidates for both SIMD optimization and GPU acceleration.
- **LatticeFold+ or Neo as a backend:** Replacing the current folding backend with LatticeFold+ [42] or Neo [43] is a natural next step. LatticeFold+ offers a more efficient folding design, while Neo is especially interesting because it targets CCS over small fields directly and introduces pay-per-bit commitments.
- **Algebraic and architectural optimization:** Beyond low-level implementation tuning, the system may benefit from more specialized lattice oriented mathematical optimizations. Another promising direction is to revisit the proving architecture itself. Instead of proving steps in a mostly inline sequential manner, a tree shaped proving structure may allow much stronger parallelization and significantly better hardware utilization.

Resumé

7.2 Motivácia práce

Diplomová práca sa venuje využitiu dôkazov s nulovou znalosťou v ekosystéme Ethereum, konkrétne návrhu post-kvantovo bezpečného ZkVM založeného na mriežkovej kryptografii. Hlavnou motiváciou práce je problém škálovateľnosti Etherea. V súčasnom modeli musia uzly siete pri spracovaní blokov opätovne vykonávať a overovať všetky transakcie. Tento prístup je síce bezpečný, ale pri väčšom zaťažení siete vedie k vysokým výpočtovým nárokom a obmedzuje celkovú priepustnosť systému.

Jedným z hlavných smerov, ako tento problém riešiť, je použitie dôkazov s nulovou znalosťou. Tie umožňujú dokázať správnosť výpočtu bez toho, aby musel overovateľ celý výpočet vykonať znova. V prostredí Ethereum sa preto objavuje myšlienka tzv. snarkifikácie, teda nahradenia priameho overovania vykonania blokov overením stručného kryptografického dôkazu. Ak je možné správnosť vykonania bloku potvrdiť jediným dôkazom, otvára sa cesta k efektívnejšiemu overovaniu, jednoduchšej škálovateľnosti a do budúcnosti aj k prirodzenejšej integrácii rollupových riešení. Širší kontext Ethereum a snarkifikácie je spracovaný v kapitole 2, konkrétne v sekciách 2.1 a 2.2. Vývoj od ZkEVM k všeobecným ZkVM, ako aj úloha rekurzie a

skladania, sú opísané v sekcii 2.3 a znázornené na obrázkoch 2.2 a 2.3.

Popri škálovateľnosti je dôležitou motiváciou aj bezpečnosť. Mnohé súčasné ZKP systémy využívajú predpoklady, ktoré nie sú odolné voči kvantovým útokom. Časť ekosystému síce smeruje k post-kvantovým riešeniam založených na hašoch, napríklad k systémom typu STARK, no tie majú často menej priaznivé vlastnosti z pohľadu veľkosti dôkazu alebo nákladov na verifikáciu. Práve preto sa v práci skúma mriežková kryptografia ako alternatíva, ktorá ponúka post-kvantovú bezpečnosť založenú na problémoch ako SVP a SIS a zároveň zachováva algebraickú štruktúru vhodnú pre konštrukciu pokročilých dôkazových systémov.

Mriežková kryptografia je pre túto prácu dôležitá aj z architektonického hľadiska. Nie je zaujímavá iba ako náhrada za klasické schémy založené na eliptických krivkách, ale aj ako základ pre skladanie dôkazov v štýle IVC. Práce ako Ajtaijove záväzky, LaBRADOR, Greyhound a najmä LatticeFold ukazujú, že mriežkové primitíva možno využiť aj pri návrhu stručných a rekurzívnych dôkazových mechanizmov. Práca preto skúma, či možno tieto prístupy použiť na vytvorenie všeobecného RISC-V ZkVM, ktorá by bola použiteľná aj pre úlohy súvisiace s dokazovaním správneho vykonania Ethereum blokov. Teoretické vlastnosti mriežkovej kryptografie sú rozobraté v sekcii 2.4 a výber použitých kryptografických primitív v návrhu riešenia v sekcii 4.2.

Ďalšou motiváciou práce je výber vhodného výpočtového modelu. Priame dokazovanie EVM je zložité, pretože EVM nebolo navrhnuté s ohľadom na efektívnu ZK aritmetizáciu. Obsahuje komplikovanejšie inštrukcie, pracuje s 256-bitovými slovami a používa model, ktorý je pre tvorbu dôkazov menej prirodzený. Preto sa v súčasnosti pozornosť presúva k všeobecným ZkVM

nad architektúrou RISC-V, ktorá je jednoduchšia, registrová a vhodnejšia na formálne zachytenie vykonania programu. V tejto práci je RISC-V zvolený ako vhodný základ na návrh systému, ktorý by spájal post-kvantovú bezpečnosť, architektúru ZkVM a potenciálne využitie v prostredí Ethereum. Porovnanie vlastností existujúcich ZkVM je uvedené v tabuľke 2.1.

Motiváciou práce je teda spojiť tri hlavné požiadavky: zlepšenie škálovateľnosti, zachovanie post-kvantovej bezpečnosti a návrh realistickej architektúry všeobecného ZkVM. Z tejto motivácie prirodzene vyplýva aj samotný problém, ktorému sa práca venuje.

7.3 Vymedzenie problému

Na základe analýzy súčasného stavu možno povedať, že oblasť ZkVM sa rýchlo rozvíja, no zároveň je veľmi rozmanitá. Existujú riešenia založené na rôznych inštrukčných sadách, rôznych modeloch aritmetizácie aj na rôznych dôkazových mechanizmoch. Napriek tomu však v literatúre ani v dostupnom open-source ekosystéme zatiaľ neexistuje praktické riešenie všeobecného RISC-V ZkVM, ktoré by bolo postavené na mriežkových primitívach a využívalo skladanie ako základ inkrementálne overiteľného výpočtu.

Práve táto medzera tvorí hlavný problém diplomovej práce. Hoci mriežková kryptografia a schémy ako LatticeFold teoreticky naznačujú, že post-kvantové skladanie dôkazov je možné, nie je zrejmé, či sa tieto myšlienky dajú pretaviť do funkčného systému pre všeobecné vykonávanie programov. Nestačí totiž iba navrhnúť jednotlivé kryptografické primitíva. Je potrebné vytvoriť celú architektúru, v ktorej bude možné reprezentovať stav virtuálneho stroja, aritmetizovať jednotlivé kroky vykonania, kontrolovať konzistenciu pamäte a spájať všetky kroky do jedného IVC reťazca. Tento problém je formulovaný

v kapitole 3. Základná schéma navrhovanej architektúry je rozpracovaná v kapitole návrhu riešenia 4, najmä v sekciách 4.1, 4.3 a 4.4, a znázornená na obrázkoch 4.1 a 4.1.

Ďalšou časťou problému je výkonnosť takéhoto riešenia. To, že je nejaký prístup teoreticky možný, ešte neznamená, že bude aj prakticky použiteľný. Pri návrhu ZkVM založeného na mriežkovej kryptografii vznikajú náklady spojené s aritmetizáciou inštrukcií, generovaním dôkazových dát, skladaním inštancií a rekurzívnym overovaním. Bez konkrétnej implementácie nie je možné reálne posúdiť, ako sa takýto systém správa, aké sú jeho časové a pamäťové nároky a ako sa porovnáva s existujúcimi ZkVM založenými na klasických alebo hash-based post-kvantových prístupoch.

Dôležitou súčasťou problému je aj otázka dlhých výpočtov. Ak má byť takýto systém zaujímavý pre Ethereum, nestačí ukázať, že funguje na malých ukážkových príkladoch. Musí byť aspoň na prototypovej úrovni overené, či je schopný zvládať dlhšie výpočtové stopy bez zjavného zlyhania. V prípade mriežkovej kryptografie je táto otázka obzvlášť dôležitá aj kvôli rastu šumu a praktickým obmedzeniam, ktoré sa môžu objaviť pri dlhšom rekurzívnom skladaní.

Problém riešený v tejto diplomovej práci možno preto zhrnúť nasledovne: chýba konkrétna a experimentálne overená inštancia všeobecného RISC-V ZkVM založeného na mriežkovej kryptografii, ktorá by umožnila posúdiť jeho realizovateľnosť, výkonnostné kompromisy a vhodnosť pre dlhšie výpočty. Práca na túto medzeru reaguje návrhom a implementáciou prototypu. Cieľom nie je vytvoriť okamžite produkčne použiteľný systém, ale overiť, či je takýto smer z architektonického a implementačného hľadiska uskutočniteľný a aké obmedzenia sa pri ňom prejavia.

7.4 Návrh riešenia

Návrh riešenia je opísaný v kapitole 4. Jeho cieľom je vytvoriť architektúru všeobecného RISC-V ZkVM, ktorá bude vhodná pre dokazovanie dlhých výpočtov a zároveň bude postavená na mriežkových kryptografických primitívach. Navrhovaný systém je rozdelený na niekoľko vrstiev. Na vstupnej strane vchádza program napísaný vo vyššom programovacom jazyku, typicky v Ruste, ktorý sa skompiluje do binárnej podoby pre 32-bitovú architektúru RISC-V. Tento binárny súbor následne vykonáva virtuálny stroj, pričom počas vykonávania vzniká stopa obsahujúca informácie o stave stroja, dekódovanej inštrukcii a prípadných pamäťových operáciách. Celkový tok spracovania je znázornený na obrázku 4.1.

Základným stavebným prvkom navrhovaného riešenia je tzv. krokový obvod F, ktorý je opísaný v sekcii 4.3. Tento obvod reprezentuje jeden krok vykonania RISC-V programu. Jeho úlohou je overiť, že prechod medzi stavom pred vykonaním inštrukcie a stavom po jej vykonaní zodpovedá pravidlám zvolenej ISA. To zahŕňa načítanie inštrukcie, jej dekódovanie, kontrolu správnosti operandov, vykonanie samotnej semantiky inštrukcie a aktualizáciu stavu stroja. Zvolený kryptografický postup pracuje nad algebraickou štruktúrou z Goldilocks cyklotomického poľa, bolo potrebné aritmetizovať 32-bitovú logiku procesora do formy vhodnej pre CCS. Práve CCS predstavuje model obmedzení, na ktorom je návrh postavený.

Samotný obvod F však nestačí na dokazovanie dlhých výpočtov. Z tohto dôvodu je v sekcii 4.4 definovaný rozšírený obvod F', ktorý slúži ako jadro IVC mechanizmu. Tento obvod obaluje jeden krok vykonania a zároveň zabezpečuje integritu celého reťazca dôkazov. V každom kroku kontroluje, že aktuálny stav nadväzuje na predchádzajúci stav, že predchádzajúci akumulátor je platný

a že predošlé skladanie prebehlo korektne. Na tento účel je do návrhu zaradený aj vnorený overovateľ pre LatticeFold. Výsledkom je architektúra, v ktorej sa jednotlivé kroky výpočtu neoverujú izolovane, ale sú postupne spájané do jedného priebežne aktualizovaného akumulátora.

Z kryptografického hľadiska je návrh riešenia založený na kombinácii viacerých stavebných blokov, ktoré sú rozobraté v sekcii 4.2. Základom je model IVC, ktorý umožňuje dokazovať správnosť dlhého výpočtu inkrementálne. Praktickú realizáciu tejto myšlienky zabezpečuje skladanie inštancií, inšpirované prácami Nova a HyperNova, no v tejto práci je klasický mechanizmus nahradený schémou LatticeFold. Tá poskytuje post-kvantové bezpečnostné vlastnosti a zároveň umožňuje budovať rekurzívny dôkazový mechanizmus nad mriežkovými predpokladmi. Dôležitou súčasťou návrhu je aj použitie Poseidon2 záväzkov na reťazenie krokov a identifikáciu stavu IVC, ako aj voľba obaľovacieho SNARKu, kde sa uvažuje o spojení SuperSpartan a Greyhound, opísané v sekcii 4.2.6. Záverečný krok obalenia výsledného artefaktu je znázornený na obrázku 4.1.

Osobitnou otázkou návrhu bola práca s pamäťou. Priamo vkladať úplnú kontrolu pamäte do každého kroku výpočtu by znamenalo výrazné zvýšenie zložitosti obvodu. Preto návrh využíva odloženú kontrolu konzistencie pamäte, opísanú v sekcii 4.2.7. Počas vykonávania sa zaznamenávajú všetky čítania a zápisy do pamäťového logu, pričom ich úplná kontrola sa presúva až do neskoršej fázy. Tento prístup znižuje záťaž krokového obvodu a zároveň zachováva možnosť overiť správnosť read-over-write správania pamäte.

Z pohľadu architektúry je dôležité, že návrh riešenia nie je iba zoznamom samostatných kryptografických techník, ale tvorí prepojený systém. Sekcia 4.1 opisuje vysokú úroveň architektúry, sekcia 4.3 formalizuje samotný krok

vykonania a sekcia 4.4 vysvetľuje, ako sa tieto kroky spájajú do dlhého dôkazového reťazca. Cieľom návrhu nie je deklarovať finálne optimalizované riešenie, ale ukázať, že je možné vytvoriť konzistentnú architektúru, v ktorej sa spoja RISC-V vykonávanie, CCS aritmetizácia, IVC, skladanie pomocou LatticeFold a post-kvantový obaľovací mechanizmus do jedného uceleného ZkVM návrhu.

7.5 Implementácia

Implementácia navrhnutého riešenia je opísaná v kapitole 5. Jej cieľom bolo vytvoriť funkčný prototyp, na ktorom bude možné overiť praktickú uskutočniteľnosť navrhnutej architektúry. Implementácia nepokrýva všetky časti, ktoré by boli potrebné pre plnohodnotný produkčný ZkVM, ale zahŕňa hlavné stavebné bloky potrebné na experimentálne overenie konceptu. Súčasťou prototypu je podmnožina inštrukcií architektúry `rv32imac`, CCS obmedzenia pre ich vykonávanie, CCS obmedzenia pre Poseidon2 záväzky a taktiež obmedzenia potrebné pre vnútro obvodového overovateľa.

Prototyp bol implementovaný v jazyku Rust a použitý technologický základ je opísaný v sekcii 5.1. Algebraická a hašovacia vrstva je postavená na knižniciach z ekosystému Plonky3, samotné skladanie využíva implementáciu LatticeFold od Nethermind. Ako hosťovský program bol v rámci experimentov použitý aj `REVM`, čo umožnilo skúmať správanie systému na dlhších výpočtoch súvisiacich s Ethereum prostredím.

Významnou súčasťou implementácie bolo vytvorenie vlastného RISC-V emulátora, ktorý generuje vykonávaciu stopu potrebnú pre dôkazový systém. Emulátor vykonáva program po jednotlivých inštrukciách a pri každom kroku sprístupňuje úplný stav stroja. Tým sa oddelila samotná exekúcia programu od

generovania dôkazových dát. Vykonávacia stopa sa následne používa pri tvorbe dôkazového vektora a pri napĺňaní CCS štruktúr. Vysokoúrovňový pohľad na implementovaný tok vykonania je znázornený na obrázku 5.1.

Z implementačného hľadiska patrili medzi najnáročnejšie časti integrácia kryptografických knižníc a syntéza obmedzení. Bežné kryptografické knižnice sú navrhnuté tak, aby vracali najmä finálny výsledok operácie, zatiaľ čo v ZkVM je potrebné zachytiť aj medzistavy výpočtu na účely witness generovania. Preto bola vytvorená vlastná implementácia permutácie Poseidon2, ktorá sprístupňuje vnútorné stavy potrebné pre CCS obmedzenia. Podobne bolo potrebné upraviť aj prácu s knižnicou LatticeFold, aby bolo možné znovu použiť interné verifierové a pomocné rutiny pri konštrukcii in-circuit logiky.

Ďalšou dôležitou časťou implementácie bola syntéza CCS obmedzení. Tá zahŕňala preklad logiky RISC-V inštrukcií do algebraickej formy, explicitné zachytenie výpočtu Poseidon2 a aj konštrukciu seba referenčných obmedzení potrebných pre overovateľa skladania. Keďže tieto časti závisia od konečnej štruktúry celého CCS, implementácia musela prebiehať vo viacerých fázach. Súčasťou práce boli aj praktické optimalizácie, napríklad zdieľanie maticových štruktúr medzi podobnými typmi obmedzení, aby zostal prototyp spustiteľný aj na bežnom hardvéri.

Výsledkom implementácie nie je úplne dokončený produkčný systém, ale funkčný prototyp hlavných architektonických komponentov. Tento prototyp umožnil overiť, že navrhnutý prístup nie je iba teoretický, ale že jeho kľúčové časti možno skutočne spojiť do jedného bežiaceho systému a následne experimentálne vyhodnotiť.

7.6 Vyhodnotenie

Vyhodnotenie prototypu je spracované v kapitole 6. Jeho cieľom nie je preukázať produkčnú pripravenosť systému, ale posúdiť, či navrhnutá architektúra funguje v praxi a aké výsledky dosahuje neúplný prototyp.

Základný benchmark bol vykonaný na jednoduchom programe `fibonacci`. V optimalizovanej binárke kompilátor výsledok 100. Fibonacciho čísla fakticky materializoval ako konštantu, takže beh nemeral plných 100 iterácií, ale krátku vykonávaciu stopu. Pri tomto behu vznikla stopa dlhá 16 inštrukcií a zaznamenali sa 2 pamäťové operácie. Celkový čas skladania dosiahol 688,24 sekundy. Už tento jednoduchý príklad ukázal, že aktuálny prototyp je výrazne pomalší než moderné ZkVM riešenia.

Podrobnejšie porovnanie s inými systémami je uvedené v tabuľke 6.1. Na rovnakom hardvéri dosiahla implementácia tejto práce čas generovania dôkazu 11:28.92 pri maximálnej spotrebe 12.34 GB RAM. Pre porovnanie, Pico dosiahlo 2:04.50 a 6.27 GB RAM, SP1 dosiahlo 1:50.17 a 9.96 GB RAM a ZKsync Airbender 4:06.73 pri 24.34 GB RAM. Z týchto výsledkov vyplýva, že navrhovaný systém je v aktuálnej podobe pomalší ako porovnávané riešenia. Jednou z možných budúcich výhod však môže byť menšia veľkosť finálneho dôkazu.

Popri malom benchmarku bol vykonaný aj dlhší experiment s hosťovským programom založeným na `REVM`. Beh trval približne 9 hodín a 29 minút a nakoniec sa zastavil kvôli chybe pri spracovaní inštrukcie, no počas behu sa IVC reťazec ďalej overoval a maximálna spotreba pamäte sa pohybovala okolo 13.7 GB. To je pozitívny signál, že veľkosť počas skladania zostala v medziach, v ktorých LatticeFold dokáže pokračovať. Hoci ani tento výsledok neznamená pripravenosť na produkčné dokazovanie Ethereum blokov,

naznačuje, že dlhšie výpočty sú realizovateľné.

Vyhodnotenie sa venuje aj otázke, prečo je aktuálny prototyp pomalší. Hlavným praktickým problémom sa ukázala byť algebraická náročnosť skladania, najmä práca s multilineárnymi rozšíreniami a súvisiacimi operáciami v rámci LatticeFold. Profilovanie ukázalo, že dominantná časť výpočtu neleží v samotnom vykonávaní virtuálneho stroja, ale v algebraických operáciách potrebných pre skladanie. Druhým možným dôvodom je samotná voľba CCS ako výpočtového modelu.

Na základe vykonaných experimentov práca odpovedá aj na tri výskumné otázky. Na otázku realizovateľnosti mriežkového RISC-V ZkVM odpovedá čiastočne kladne: na architektonickej úrovni bol takýto systém navrhnutý a na prototypovej úrovni aj implementovaný, hoci zatiaľ neúplne. Na otázku výkonových kompromisov je odpoveď v súčasnom stave skôr negatívna, keďže implementácia je pomalšia než porovnávané ZkVM. Na otázku vhodnosti pre dlhé výpočty sú výsledky opatrne pozitívne, pretože dlhší experiment neukázal okamžitý problém s nekontrolovaným rastom pamäte ani zlyhanie rekurzívneho reťazca, hoci na definitívne závery by bolo potrebné rozsiahlejšie testovanie.

Celkové vyhodnotenie teda ukazuje, že navrhovaný smer je architektonicky uskutočniteľný, ale zatiaľ výkonovo nekonkurencieschopný voči etablovaným riešeniam. Prototyp však potvrdil, že mriežková kryptografia sa dá použiť ako základ pre všeobecný RISC-V ZkVM a že tento smer má zmysel ďalej rozvíjať.

7.7 Záver

Záver práce je spracovaný v kapitole 7. Diplomová práca sa venovala návrhu a prototypovej implementácii post-kvantovo bezpečného RISC-V ZkVM založeného na mriežkovej kryptografii s potenciálnym využitím v prostredí Ethereum. Hlavným prínosom práce je návrh ucelenej architektúry, ktorá spája RISC-V vykonávanie, CCS aritmetizáciu, IVC, skladanie pomocou LatticeFold a uvažovaný obaľovací mechanizmus založený na kombinácii SuperSpartan a Greyhound.

Práca zároveň ukázala, že tento smer nie je iba teoretickou úvahou. Podarilo sa implementovať funkčný prototyp hlavných stavebných blokov systému, ktorý umožnil experimentálne overiť základnú realizovateľnosť navrhovaného prístupu. Výsledky síce ukazujú, že aktuálna implementácia zaostáva za etablovanými ZkVM z pohľadu výkonu a nemožno ju považovať za produkčne pripravené riešenie, no zároveň naznačujú, že hlavné obmedzenia sú skôr implementačné a aritmetizačné ako principiálne.

Z pohľadu výskumného prínosu je dôležité najmä to, že práca adresuje konkrétnu medzeru v súčasnom stave poznania. Ukazuje, ako možno mriežkové primitíva využiť pri návrhu všeobecného ZkVM a aké praktické problémy pri tom vznikajú. Zároveň poskytuje prvé experimentálne údaje, na základe ktorých možno realistickejšie hodnotiť výkonové kompromisy, vhodnosť zvoleného výpočtového modelu a správanie systému pri dlhších výpočtoch.

Práca tým vytvára základ pre ďalší výskum. Prirodzenými pokračovaniami sú efektívnejší výpočtový model než CCS, lepšia algebraická optimalizácia, využitie SIMD alebo GPU akcelerácie a tiež prechod na novšie mriežkové schémy, napríklad LatticeFold+ alebo Neo. Celkovo možno uzavrieť, že navrhnutý smer je perspektívny a zaslúži si ďalšie rozpracovanie, aj keď

v súčasnom stave ešte nepredstavuje konkurencieschopné produkčné riešenie.

Use of AI

AI was used in document writing, implementation, and evaluation.

7.8 Document writing

- brainstorming how to best write and explain certain parts,
- proofreading,
- cross-checking citations,
- writing LaTeX commands and TikZ diagrams.

This was controlled by reading the generated corrections, opening the cited papers at the reported places, and checking the compiled document and diagrams visually.

7.9 Implementation

- debugging CCS satisfiability in SageMath,
- writing tests for correct RISC-V VM execution and instruction decoding,
- navigating the forked dependencies,

- writing VM instruction handlers,
- setting up the EVM guest program for proving block 24740084, including downloading all necessary block, transaction, and Ethereum state data from the Ethereum blockchain.

This was controlled by changing constraints until the numbers in the constraints matched the numbers from the Sage scripts and the checks passed, running the tests on compiled guest binaries to validate the VM instruction handlers, opening the reported dependency sources to verify that the guidance matched the code, and running REVM on the downloaded data to validate block 24740084.

7.10 Evaluation

- researching how other zkVMs are run.

This was controlled by actually running the referenced systems and checking that the described commands and observed behaviour matched.

References

[1] *Ethereum Whitepaper — ethereum.org*. https://ethereum.org/en/whitepaper/. [Accessed 28-05-2025].

[2] Vitalik Buterin et al. *Combining GHOST and Casper*. 2020. DOI: 10.48550/ARXIV.2003.03052. URL: https://arxiv.org/abs/2003.03052.

[3] *Consensus mechanisms — ethereum.org*. https://ethereum.org/en/developers/docs/consensus-mechanisms/. [Accessed 29-05-2025].

[4] *Lean Consensus Roadmap — leanroadmap.org*. https://leanroadmap.org/. [Accessed 10-12-2025].

[5] *Native rollups—superpowers from L1 execution — ethresear.ch*. https://ethresear.ch/t/native-rollups-superpowers-from-l1-execution/21517. [Accessed 31-05-2025].

[6] *RISC-V Ratified Specifications — riscv.org*. https://riscv.org/specifications/ratified/. [Accessed 01-06-2025].

[7] *Untitled :: RISC-V Ratified Specifications L ibrary — docs.riscv.org*. https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html. [Accessed 04-05-2026].

[8] Yunbo Yang et al. *SoK: Understanding zkVM: From Research to Practice*. Cryptology ePrint Archive, Paper 2026/525. 2026. URL: https://eprint.iacr.org/2026/525.

[9] Eli Ben-Sasson et al. *Scalable, transparent, and post-quantum secure computational integrity*. Cryptology ePrint Archive, Paper 2018/046. 2018. URL: https://eprint.iacr.org/2018/046.

[10] Miklós Ajtai. "The shortest vector problem in L2 is NP-hard for randomized reductions (extended abstract)". In: *Proceedings of the thirtieth annual ACM symposium on Theory of computing - STOC '98*. STOC '98. ACM Press, 1998, 10–19. DOI: 10.1145/276698.276705. URL: http://dx.doi.org/10.1145/276698.276705.

[11] Yang Li, Kee Siong Ng, and Michael Purcell. *A Tutorial Introduction to Lattice-based Cryptography and Homomorphic Encryption*. 2022. DOI: 10.48550/ARXIV.2208.08125. URL: https://arxiv.org/abs/2208.08125.

[12] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plancon. *Lattice-Based Zero-Knowledge Proofs and Applications: Shorter, Simpler, and More General*. Cryptology ePrint Archive, Paper 2022/284. 2022. URL: https://eprint.iacr.org/2022/284.

[13] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf. [Accessed 28-05-2025].

[14] *Ethereum Virtual Machine (EVM) — ethereum.org*. https://ethereum.org/en/developers/docs/evm/. [Accessed 29-05-2025].

[15] Oded Goldreich, Silvio Micali, and Avi Wigderson. "Proofs that yield nothing but their validity or all languages in NP have zero-knowledge

proof systems". In: *Journal of the ACM (JACM)* 38.3 (1991), pp. 690–728.

[16] Amos Fiat and Adi Shamir. "How To Prove Yourself: Practical Solutions to Identification and Signature Problems". In: *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, pp. 186–194. ISBN: 9783540180470. DOI: 10.1007/3-540-47721-7_12. URL: http://dx.doi.org/10.1007/3-540-47721-7_12.

[17] Nir Bitansky et al. *From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again*. Cryptology ePrint Archive, Paper 2011/443. 2011. URL: https://eprint.iacr.org/2011/443.

[18] - *YouTube — youtube.com*. https://www.youtube.com/watch?v=lRqnFrqpq4k. [Accessed 31-05-2025].

[19] *The different types of ZK-EVMs — vitalik.eth.limo*. https://vitalik.eth.limo/general/2022/08/04/zkevm.html. [Accessed 01-06-2025].

[20] Lior Goldberg, Shahar Papini, and Michael Riabzev. *Cairo – a Turing-complete STARK-friendly CPU architecture*. Cryptology ePrint Archive, Paper 2021/1063. 2021. URL: https://eprint.iacr.org/2021/1063.

[21] Eli Ben-Sasson et al. "Fast Reed-Solomon Interactive Oracle Proofs of Proximity". In: *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Ed. by Ioannis Chatzigiannakis et al. Vol. 107. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, 14:1–14:17. ISBN: 978-3-95977-076-7. DOI: 10.4230/LIPIcs.ICALP.2018.14. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14.

[22] Jens Groth. *On the Size of Pairing-based Non-interactive Arguments*. Cryptology ePrint Archive, Paper 2016/260. 2016. URL: https://eprint.iacr.org/2016/260.

[23] Paul Valiant. "Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency". In: *Theory of Cryptography*. Springer Berlin Heidelberg, 1–18. ISBN: 9783540785248. DOI: 10.1007/978-3-540-78524-8_1. URL: http://dx.doi.org/10.1007/978-3-540-78524-8_1.

[24] Abhiram Kothapalli and Srinath Setty. *SuperNova: Proving universal machine executions without universal circuits*. Cryptology ePrint Archive, Paper 2022/1758. 2022. URL: https://eprint.iacr.org/2022/1758.

[25] Eli Ben-Sasson et al. *Scalable Zero Knowledge via Cycles of Elliptic Curves*. Cryptology ePrint Archive, Paper 2014/595. 2014. URL: https://eprint.iacr.org/2014/595.

[26] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Paper 2019/953. 2019. URL: https://eprint.iacr.org/2019/953.

[27] *plonky2/plonky2/plonky2.pdf at main · 0xPolygonZero/plonky2 — github.com*. https://github.com/0xPolygonZero/plonky2/blob/main/plonky2/plonky2.pdf. [Accessed 01-06-2025].

[28] *halo2 - The halo2 Book — zcash.github.io*. https://zcash.github.io/halo2/index.html. [Accessed 01-06-2025].

[29] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. *Nova: Recursive Zero-Knowledge Arguments from Folding Schemes*. Cryptology ePrint Archive, Paper 2021/370. 2021. URL: https://eprint.iacr.org/2021/370.

[30] Abhiram Kothapalli and Srinath Setty. *HyperNova: Recursive arguments for customizable constraint systems*. Cryptology ePrint Archive, Paper 2023/573. 2023. URL: https://eprint.iacr.org/2023/573.

[31] *Ethproofs | Ethproofs — ethproofs.org*. https://ethproofs.org/. [Accessed 02-06-2025].

[32] *Design and Specifications — docs.openvm.dev*. https://docs.openvm.dev/specs/openvm/overview/. [Accessed 04-05-2026].

[33] *SP1 Hypercube: Proving Ethereum in Real-Time — blog.succinct.xyz*. https://blog.succinct.xyz/sp1-hypercube/. [Accessed 04-05-2026].

[34] M. Ajtai. "Generating hard instances of lattice problems (extended abstract)". In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*. STOC '96. Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, 99–108. ISBN: 0897917855. DOI: 10.1145/237814.237838. URL: https://doi.org/10.1145/237814.237838.

[35] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. "On Ideal Lattices and Learning with Errors over Rings". In: *Journal of the ACM* 60.6 (Nov. 2013), 1–35. ISSN: 1557-735X. DOI: 10.1145/2535925. URL: http://dx.doi.org/10.1145/2535925.

[36] Sedat Akleylek et al. *An Efficient Lattice-Based Signature Scheme with Provably Secure Instantiation*. Cryptology ePrint Archive, Paper 2016/030. 2016. URL: https://eprint.iacr.org/2016/030.

[37] Chris Peikert. *Public-Key Cryptosystems from the Worst-Case Shortest Vector Problem*. Cryptology ePrint Archive, Paper 2008/481. 2008. URL: https://eprint.iacr.org/2008/481.

[38] Adeline Langlois and Damien Stehle. *Worst-Case to Average-Case Reductions for Module Lattices*. Cryptology ePrint Archive, Paper 2012/090. 2012. URL: https://eprint.iacr.org/2012/090.

[39] Ward Beullens and Gregor Seiler. *LaBRADOR: Compact Proofs for R1CS from Module-SIS*. Cryptology ePrint Archive, Paper 2022/1341. 2022. URL: https://eprint.iacr.org/2022/1341.

[40] Ngoc Khanh Nguyen and Gregor Seiler. *Greyhound: Fast Polynomial Commitments from Lattices*. Cryptology ePrint Archive, Paper 2024/1293. 2024. DOI: 10.1007/978-3-031-68403-6_8. URL: https://eprint.iacr.org/2024/1293.

[41] Dan Boneh and Binyi Chen. *LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems*. Cryptology ePrint Archive, Paper 2024/257. 2024. URL: https://eprint.iacr.org/2024/257.

[42] Dan Boneh and Binyi Chen. *LatticeFold+: Faster, Simpler, Shorter Lattice-Based Folding for Succinct Proof Systems*. Cryptology ePrint Archive, Paper 2025/247. 2025. URL: https://eprint.iacr.org/2025/247.

[43] Wilson Nguyen and Srinath Setty. *Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments*. Cryptology ePrint Archive, Paper 2025/294. 2025. URL: https://eprint.iacr.org/2025/294.

[44] Srinath Setty, Justin Thaler, and Riad Wahby. *Customizable constraint systems for succinct arguments*. Cryptology ePrint Archive, Paper 2023/552. 2023. URL: https://eprint.iacr.org/2023/552.

[45] Alex Ozdemir, Evan Laufer, and Dan Boneh. *Volatile and Persistent Memory for zkSNARKs via Algebraic Interactive Proofs*. Cryptology ePrint Archive, Paper 2024/979. 2024. URL: https://eprint.iacr.org/2024/979.

[46] Srinath Setty. *Spartan: Efficient and general-purpose zkSNARKs without trusted setup*. Cryptology ePrint Archive, Paper 2019/550. 2019. URL: https://eprint.iacr.org/2019/550.

[47] Lorenzo Grassi et al. *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. Cryptology ePrint Archive, Paper 2019/458. 2019. URL: https://eprint.iacr.org/2019/458.

[48] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. *Poseidon2: A Faster Version of the Poseidon Hash Function*. Cryptology ePrint Archive, Paper 2023/323. 2023. URL: https://eprint.iacr.org/2023/323.

[49] Carsten Lund et al. "Algebraic methods for interactive proof systems". In: *Journal of the ACM* 39.4 (Oct. 1992), 859–868. ISSN: 1557-735X. DOI: 10.1145/146585.146605. URL: http://dx.doi.org/10.1145/146585.146605.

[50] Srinath Setty, Justin Thaler, and Riad Wahby. *Unlocking the lookup singularity with Lasso*. Cryptology ePrint Archive, Paper 2023/1216. 2023. URL: https://eprint.iacr.org/2023/1216.